MEGASCRIPT DIGITAL

AI Website Platform

Technical Design Document (TDD)

A build blueprint for a conversational, template-driven website platform with managed hosting, domain automation, AI content generation and portable exports.

Document version: 1.0 - Baseline architecture
Prepared for: MegaScript Digital
Status: Implementation blueprint - decisions requiring validation are flagged
Date: 14 October 2026
Working product name: Fortress / Fortress AI (provisional; not legally cleared)

CORE DESIGN PRINCIPLE The canonical website is structured configuration plus versioned content and assets. A shared renderer serves managed sites; a separate export pipeline compiles that same configuration into portable static files.

Document Control

Field	Value
Owner	MegaScript Digital product/engineering owner
Audience	Product owner, technical lead, application developers, UI/UX designers, QA, DevOps, support and future contractors
Scope	MVP through a production-ready first release, including architecture decisions and later-phase options
Assumptions	Web-first launch; mobile-first responsive UI; multi-tenant shared renderer; AI-generated content; curated licensed imagery; hosted and static-export delivery
Naming	“Fortress” is used as a temporary label only. Name, domain, social handles and trademark clearance remain open.
Vendor status	Vercel/Cloudflare, Cloudinary, Brevo, IntaSend, registrar and model providers are candidates; pricing, limits and contracts must be validated before launch.
Change control	Architecture decisions and requirement changes should be recorded as ADRs and versioned product requirements.

Revision History

Version	Date	Change
0.1	Concept stage	Product concept and feature inventory
1.0	14 Oct 2026	Baseline TDD covering requirements, architecture, workflows, data, APIs, security, deployment, billing, testing and roadmap

How to Use This Document

This TDD is intended to be actionable, not a promise that every external vendor capability is already contracted or enabled. Implementers should convert each requirement into a ticket with acceptance tests, keep schemas and API contracts in version control, and resolve items marked “validate” before production. Where a product choice is provisional, the design describes an adapter so that the provider can be replaced without rewriting the core product.

Contents Overview

- 1 Executive Summary and Product Definition
- 2 Goals, Non-Goals and Product Principles
- 3 Personas, Roles and User Journeys
- 4 Functional Scope and MVP Boundary
- 5 System Context and Logical Architecture
- 6 Technology Decisions and ADRs
- 7 Multi-Tenant Site Model and Isolation
- 8 Website Configuration and Composition Engine
- 9 Section Registry and Manifest Contract
- 10 Theme, Typography and Design Tokens
- 11 Multi-Stage AI Orchestration
- 12 Prompt Contracts and Content Quality
- 13 Conversation and Incremental Editing
- 14 Cloudinary Asset Catalogue and Selection
- 15 Media Representation and Regional Context
- 16 Site Pages and Standard Components
- 17 Blog, Content Management and SEO

- 18 GSAP Animation System
- 19 Responsive and Accessible Design
- 20 Hosted Publishing and Routing
- 21 Static Export and Portability
- 22 Domains, DNS, TLS and Registrar Adapter
- 23 Authentication, Sessions and Verification
- 24 Billing, Plans, Usage and Payments
- 25 Email and Notification Architecture
- 26 Database Design and Data Lifecycle
- 27 API Design and Webhooks
- 28 Background Jobs, Retries and Idempotency
- 29 Security, Abuse Prevention and Secrets
- 30 Privacy, Consent and Governance
- 31 Performance and Reliability Targets
- 32 Observability, Audit and Support
- 33 Deployment, CI/CD and Environments
- 34 Testing and Release Gates
- 35 Operations, Backup and Disaster Recovery
- 36 Delivery Roadmap and Acceptance
- 37 Functional Requirement Specifications FR-001 to FR-090
- 38 Data Dictionary and Relationship Notes
- 39 API and Event Catalogues
- 40 Reference Configurations and Prompt Examples
- 41 Risk Register and Open Decisions
- 42 Runbooks, Checklists and Glossary
- 43 Source References

Pagination is generated by Word at render time. The sections above are intentionally descriptive rather than hard-coded to page numbers.

1. Executive Summary and Product Definition

The platform converts MegaScript Digital's professional website-building methods into a repeatable, self-service product. The customer describes their business through a guided conversational interface; an orchestration service plans a supported site; multiple focused AI jobs draft content; a composition engine combines registered components, themes and curated assets; and the customer previews, edits, pays for, publishes and manages the result in one account.

1.1 Product statement

Working proposition: "A professional website for your business, built and managed through a simple conversation." The customer should not need to understand hosting, DNS, server deployments, React, CSS, sitemap generation or image optimisation to launch a site. The product should make informed defaults, ask only questions that materially affect the result, and expose advanced controls only when helpful.

1.2 Core capabilities

- Conversational intake with Google OAuth or email/password signup, email verification and optional phone verification.
- Generation of a multipage business website: Home, About, Services/Products, Blog and Contact, with industry-specific additions where appropriate.
- A modular registry of professionally engineered section components, each versioned and defined by content schemas, layout rules, responsive behaviour, media slots and animation presets.
- Multi-stage AI generation with a site-plan stage, page-level content stages, separately generated launch blog drafts, schema validation and targeted retries.
- A curated Cloudinary-backed image catalogue with application-level metadata, licensing records, responsive transformations and fallbacks.
- Hosted publishing using a shared multi-tenant renderer, domain mapping, TLS, SEO artefacts and versioned release/rollback.
- Portable static export of pages and local assets, plus an explicit dependency report for functionality that requires a backend.
- Monthly and annual plans with a defined AI allowance, separately visible domain registration/renewal charges and supported top-ups or upgrades.
- Brevo-powered transactional email, notifications, renewal reminders and opt-in platform newsletters.
- Mobile-first dashboard: desktop sidebar, mobile bottom navigation, a top notification control, Home, Build, Websites and Profile destinations.

1.3 Product success measures

Dimension	Initial measurement	Interpretation
Activation	Share of verified accounts that finish a first preview	Measures onboarding clarity and whether intake is sufficiently lightweight.
Time to first preview	Median time from completed business brief to preview-ready version	Tracks orchestration speed and job reliability; not a promise of "minutes" for every case.
Publish conversion	Share of preview-ready sites that publish within an agreed window	Shows perceived quality and clarity of publishing/pricing.
Build quality	Automated checks passed plus rubric-based human review on sample sites	Prevents speed from replacing professional quality.
Edit success	Share of targeted edit jobs that apply without breaking other sections	Validates patch-based editing rather than full regeneration.
Reliability	Successful publish/domain operations and recovery time	Measures operational health.
Unit economics	Hosting + image delivery + AI + payment + domain support cost per site vs revenue	Protects gross margin.
Portability	Successful static export test across supported host types	Ensures export is real, usable independence rather than a marketing claim.

1.4 Core operating principle

CANONICAL SOURCE OF TRUTH Persist structured site configuration and content revisions. Do not make generated HTML or a transient model response the only record of the website. Hosted rendering and portable export must both be deterministic projections of an approved configuration version.

2. Goals, Non-Goals and Product Principles

The MVP should solve the common case exceptionally well: a credible business website with good content, robust mobile layouts, SEO foundations and reliable publishing. It should not attempt to recreate every feature of a bespoke agency project on day one.

2.1 Goals

- Make professional multipage websites usable by nontechnical business owners.
- Turn the owner's business facts into coherent text using section-specific instructions, not arbitrary AI-generated source code.
- Create controlled visual variety through section variants, supported themes, curated imagery and weighted selection.
- Keep creation, editing, hosting, domains, billing and customer communications in one managed interface.
- Allow the customer to export a standard static site and use another host, while transparently identifying dynamic dependencies.
- Support future growth in industries, sections, integrations, plan types and AI providers without redesigning the platform.
- Use mobile-first design and accessibility as engineering constraints, not final polish.

2.2 Non-goals for initial release

- Unlimited arbitrary code generation or an unrestricted visual canvas.
- A full ecommerce engine with carts, inventory, payments, shipping and tax workflows.
- Complex booking systems, portals, memberships or custom business logic in the initial MVP.
- Guaranteed Google rankings, indexed pages, leads or revenue.
- Cross-site similarity scans during every composition; diversity is handled by deterministic rules and within-site validation.
- Automatic synchronisation of an exported website unless the customer has connected a supported external deployment.
- Customer-managed API keys for AI models. The platform manages approved provider credentials server-side.
- Running customer-specific application deployments by default; the first architecture is shared-renderer/multi-tenant.

2.3 Product principles

Principle	Engineering interpretation
Good defaults, minimal questions	Reuse intake data; ask follow-ups only when content or risk materially depends on the answer.
Constrained AI	Models select from registered options and fill schemas; runtime code owns validation and side effects.
Composable design	Section code is versioned; site instances reference a component version and content payload.
Safe publishing	Preview first; immutable publish versions; domain ownership/verification; rollout checks and rollback.
Portable by design	Export generator reads the canonical config; vendor-specific features are explicitly reported.
Honest billing	Show renewal and usage rules before purchase; no assumed unlimited use or unconfirmed payment capability.
Privacy by design	Collect only necessary personal data; regional image preferences never infer visitor ethnicity.
Measure before scaling	Instrument token costs, media delivery, traffic, build steps, errors and support burden.

3. Personas, Roles and User Journeys

The interface is designed for nontechnical business owners first, with enough transparency for agencies and technically skilled customers. Internal operators need controls to maintain the component library and resolve exceptional failures without giving the end customer unnecessary complexity.

3.1 Personas

Persona	Primary needs	Likely friction	Design response
Small business owner	Launch a credible site, obtain enquiries, update business facts	Technical terms, indecision, unclear costs	Plain-language chat, sensible defaults, visible preview and clear plan.
Solo professional	Portfolio/services page, personal brand, custom domain	Limited time and content	Short intake, industry-specific prompts, reusable sections.
Agency/developer	Generate quickly, export assets/source, control domain/deployment	Black-box lock-in	Static export, version history, data portability and later source export.
Returning owner	Make a small change safely	Fear of breaking a live site	Targeted edits, staged preview, revision history and rollback.
Platform administrator	Maintain providers, sections, assets, pricing and support	High blast radius	Role-scoped dashboard, audit log, staged rollout and least privilege.
Content/asset curator	Maintain image quality, relevance and licence details	Metadata drift and expired rights	Structured metadata, review queue and licence/expiry fields.

3.2 Primary journey: first website

1. Visitor opens product marketing site and understands core value, pricing and domain terms.
2. Visitor creates an account with Google or email/password. Email verification completes before publication and domain registration are enabled.
3. User enters Home dashboard and selects Build.
4. Conversational intake records business name, industry, location/market, services, audience, preferred style, logo/brand colours if supplied, contact links and required compliance details.
5. Planner creates a sitemap and proposed composition from registered sections; only high-impact unknowns trigger follow-up questions.
6. Parallel or dependency-aware jobs generate page content and three initial blog drafts (draft status by default).
7. Asset matcher resolves curated Cloudinary assets for each manifest media slot.
8. Renderer creates preview on a platform subdomain; automated checks run.
9. User requests specific changes through chat, approves the preview and selects a plan/domain.
10. Payment verification triggers publishing/domain workflow; the system reports verified stages.
11. Website goes live; renewal reminders, usage display, backup and monitoring continue.

3.3 Return editing journey

A returning user selects a website, enters Build/Edit, and makes a natural-language request. The request router classifies the change as content patch, asset replacement, supported style token update, section insert/remove/reorder, page addition or unsupported/dynamic feature request. A scoped job updates only affected configuration, produces a preview revision, validates dependencies and asks for approval where the change has material impact. Publishing never happens silently for a destructive or high-impact change unless the owner has explicitly configured auto-publish.

3.4 Roles

Role	Permissions
Anonymous visitor	Browse product marketing pages and published customer sites; submit permitted public forms.
Customer	Manage own account, sites, content, drafts, domains, billing and notifications subject to plan and status.

Role	Permissions
Support operator	View operational metadata and support tickets; any temporary content access must be justified, scoped and audited.
Content/asset curator	Manage assets, licensing and tags; no access to billing secrets or unrelated personal profiles.
Platform administrator	Manage plans, provider configs, feature flags, section versions and operational incidents; privileged actions audited.
Automation service identity	Narrowly scoped permissions for its workflow (AI job, domain operation, billing webhook, export job); never a general user session.

4. Functional Scope and MVP Boundary

The scope prioritises a reliable core product over the largest possible feature list. Each release should be independently useful and should not require unbuilt ecommerce or booking features to work.

4.1 MVP included

- User registration/login and verified email; Google sign-in if OAuth configuration is available.
- Dashboard and conversational builder responsive on mobile, tablet and desktop.
- One site per base plan initially (limit configurable), with Home, About, Services/Products, Blog and Contact pages.
- Section registry and a deliberately curated starter set of variants; do not ship hundreds before the quality rubric is proven.
- The standard site chrome: sticky header, responsive navigation, four-column desktop footer collapsing gracefully on smaller screens, floating WhatsApp contact control if supplied, contact forms and CTA patterns.
- Theme tokens, logo upload/reference, palette/font options, image catalogue selection, reduced-motion-safe GSAP presets.
- Staged AI jobs to plan, draft page content, draft three blog posts, validate and build previews.
- Hosted publishing on platform subdomain and a first custom-domain connection path.
- Static export of content pages and assets, with dependency report and README.
- Monthly/annual plan records, payment checkout integration, usage ledger and domain renewal records.
- Transactional email and in-app notifications; opt-in newsletter.
- Administrative functions for users, sites, assets, section metadata, prompt versions, jobs, billing status and audit events.

4.2 Later phases

- Connected Git/third-party deployment and automated push publishing.
- React/TypeScript source export with package lock/build instructions.
- Search Console authorisation and analytics dashboards.
- Multiple websites per account, agency workspaces and collaborator access.
- Dedicated e-commerce, booking, membership, multilingual and advanced forms modules.
- Template marketplace and third-party component certification.
- Automated A/B test variants, traffic-based performance suggestions and other optimisation features.

4.3 Product limits should be data-driven

Page limits, AI allowances, number of sites, export entitlement and hosting quotas should be represented in plan configuration rather than scattered conditionals. Every limit needs a user-visible explanation, an API-side enforcement check and an administrative override path with an audit trail. Plan enforcement must not break access to account data, invoices, domain transfer information or export pathways where the customer is entitled to them.

5. System Context and Logical Architecture

Use a modular monolith for the first release, with clear module boundaries and background workers for long-running workflows. Avoid deploying every domain concept as a separate microservice until workload or team size proves that split is worthwhile.

5.1 Logical context

```
Customer browser (mobile / desktop)
  | HTTPS
```

```
  v
Next.js web application
  |-- Identity / session guard
  |-- Tenant/site query layer
  |-- Conversation + site editor UI
  |-- Public host resolver + shared site renderer
  |-- API routes / server actions (typed contracts)
  |
  +--> PostgreSQL (Supabase-managed) + Prisma
  +--> Workflow engine (Inngest or equivalent)
  |      |--> AI provider adapter(s)
  |      |--> Cloudinary asset resolver
  |      |--> Build / static export worker
  |      |--> Registrar adapter / DNS provider
  |      |--> Payment / email notifications
  +--> Hosting/platform APIs (custom domains, TLS, deploy status)
  +--> Observability (logs, metrics, tracing, alerts)
```

5.2 Module boundaries

Module	Owns	Must not own
Identity and access	Sessions, roles, account verification and access policy	Site generation prompt logic or payment settlement rules
Site management	Site profile, slug, lifecycle state and owner relationship	Provider credentials or registrar-side truth
Design system	Section manifest registry, themes and compatibility rules	Unvalidated arbitrary HTML from model output
AI orchestration	Prompt versions, job state, provider adapter, schema validation and usage	Direct unverified publishing side effects
Asset catalogue	Asset records, licences, tags, Cloudinary references and image selection	Assumptions about users' ethnicity or undocumented rights
Publishing/export	Immutable build inputs, build output, dependency report and release pointer	Billing settlement or model prompts
Domain lifecycle	Domain ownership, verification, registrar/DNS state, TLS readiness	Assumption that checkout success means DNS is ready
Billing	Plans, subscription state, invoices, webhook processing and usage allowance	Unverified client-side payment state
Messaging	Transactional emails, in-app notifications and consented marketing	Authentication truth or billing truth
Admin/operations	Scoped management, audit trails and incident controls	Unlogged access or broadly reusable secrets

5.3 Deployment topology

The preferred first topology is one Next.js application plus durable background jobs and managed PostgreSQL. The same application serves dashboard routes and public customer domains, resolved by hostname to a site ID. Domain hosting provider choice remains provisional until traffic profiles, custom-domain counts, commercial terms and export strategy have been modelled. A separate static export worker may use a controlled build sandbox rather than running arbitrary customer-supplied code in the web request process.

5.4 Trust boundaries

- Untrusted browser input enters through authenticated, schema-validated endpoints.
- Model output is untrusted until schema, content policy, asset ID and component ID validation succeeds.
- Webhook requests are untrusted until signature/authenticity checks pass and idempotency is applied.
- Public custom-domain requests must resolve only to verified domain mappings; unknown hosts do not default to a customer tenant.
- Admin operations and provider credentials remain behind server-side access boundaries.
- Static exports are artifacts: dynamic external service dependencies must be disclosed rather than silently assumed.

6. Technology Decisions and Architecture Decision Records

Technology is selected to make quality, portability and maintainability possible. Provider decisions should be backed by a documented proof of concept and cost comparison before commitment.

Decision	Baseline	Rationale	Validation / alternative
Web application	Next.js + React + TypeScript	Shared rendering, server routes, ecosystem and static-export capability	Verify the exact Next.js features used support export; keep an export-safe subset.
Styling	Tailwind CSS + CSS variables	Speed and consistent tokens with per-site themes	Use custom components for brand-quality public sites; UI kit is mainly for dashboard.
Database	PostgreSQL via Supabase	Relational model fits sites/pages/versions/billing/workflows	Review compute/connection pool limits, backups and RLS policy model.
ORM	Prisma	Typed data access and controlled migrations	Avoid parallel undocumented SQL schema changes; test RLS and raw-query paths.
AI	Provider adapter; one primary provider for launch	Consistent provider interface, schemas, usage telemetry and fallbacks	Benchmark outputs/costs on real briefs before selecting default; avoid premature multi-provider fanout.
Structured output	Zod runtime validation + provider native structured output where available	Machine-checkable schema; rejects invalid responses	Do not treat JSON validity as factual accuracy.
Workflow engine	Inngest or equivalent durable workflow	Retries, step persistence and resuming completed work	Prototype timeout/retry/cost behaviour; alternative: managed queue + worker with equivalent guarantees.
Asset delivery	Cloudinary + asset records in Postgres	Central curation, transformations, tags and stable identifiers	Confirm plan pricing, backup, licence workflow and transformed URL policy.
Animation	GSAP preset registry	High-quality motion with reusable controls	Keep optional, reduced-motion aware and safe under SSR/hydration.
Managed hosting	Vercel for Platforms candidate	Custom domains and multi-tenant delivery model	Confirm commercial limits/pricing; evaluate Cloudflare for SaaS or own container/VPS later.
Email	Brevo candidate	Transactional and marketing functions	Verify sender authentication, template/event APIs and regional delivery requirements.
Payments	IntaSend candidate	Kenyan-friendly M-Pesa and card collection options	Validate settlement, refunds, recurring plans by payment method, webhook and fees.
Domain registrar	Registrar adapter; ResellerClub or other suitable API candidate	Automate lifecycle and retain ability to change vendors	Confirm `.co.ke` API availability, eligibility, price/renewal, registrant rights and support.
Auth	Better Auth candidate	Google OAuth and email/password flow	Test email verification, account linking, MFA needs, rate limiting and session revocation.
Source hosting	Git-based CI/CD	Reproducible builds, code review, rollback and environment controls	Select repository provider and secrets model; require deployment approvals for production.

6.1 ADR format

Every significant technical choice should have an Architecture Decision Record: ID, date, context, options considered, decision, consequences, security implications, rollback path and trigger for reconsideration. Do not let product-specific assumptions live only in chat history.

6.2 Proposed ADR register

ADR	Topic	Status
ADR-001	Shared multi-tenant renderer as default hosting architecture	Accepted for MVP, validate provider economics
ADR-002	Canonical site configuration used by both renderer and exporter	Accepted
ADR-003	AI returns structured content/configuration, not arbitrary code	Accepted
ADR-004	Versioned section manifests and component registry	Accepted
ADR-005	Staged generation jobs and targeted retries	Accepted
ADR-006	Cloudinary-backed curated asset library	Accepted direction, contract/pricing to validate
ADR-007	Monthly/annual subscription with included AI allowance	Accepted direction; exact plans TBD
ADR-008	External hosting starts with static export; connected deploy later	Accepted phased scope
ADR-009	Fortress/Fortress AI remain provisional product names	Open
ADR-010	Identity and payment vendors	Proof-of-concept and contractual validation pending

7. Multi-Tenant Site Model and Isolation

Each customer site is logically independent even though public pages share a renderer and runtime. Tenant isolation is a security requirement and must be enforced at both service and database boundaries where feasible.

7.1 Tenant resolution

12. Parse and normalise the HTTP Host header through trusted framework handling; reject malformed, unexpected or untrusted forwarded-host values.
13. Resolve hostname to a verified DomainMapping or approved platform subdomain. Unknown hostnames route to a safe not-found page, never to a guessed tenant.
14. Resolve the canonical site ID and site publication pointer using an allowlisted host mapping.
15. Read only the published version of the site, its page content and asset references; do not expose draft content to public requests.
16. Cache by canonical site ID plus published version. Purge or rotate keys on publication and domain transfer.

7.2 Isolation invariants

- Every user-owned query includes a tenant/site ownership predicate or uses a verified access scope object.
- A `siteId` supplied by the browser is not proof of ownership.
- Domain ownership verification occurs before mapping is activated.
- Asset write credentials and provider API keys are server-only; browser upload flows use short-lived, scoped upload signatures if direct upload is supported.
- Support/admin cross-tenant access requires explicit privileged permission and must be logged.
- Background jobs carry tenant/site identity, minimal payload and an authenticated internal job ID.
- Tenant IDs are included in logs and metrics only where this does not leak customer-sensitive content.

7.3 Shared vs isolated resources

Shared infrastructure	Per-tenant logical state
Next.js application and section component code	Business profile, site config and page content
Approved global section registry	Selected sections and component versions for each site
Cloudinary account/catalogue, subject to account security	Asset selections and any customer-uploaded assets/permissions
Workflow runner and AI provider adapter	AI usage ledger, generation jobs and prompt inputs restricted to that site
Database cluster and schemas	Rows guarded by ownership/RLS policies
Host resolver and platform domain	Custom domain mappings and verification challenges
Email delivery infrastructure	Message recipients, notification preferences and account relationship

7.4 Tenant-level limits

Rate limits and quotas should be applied at account, site, IP/network and operation levels as appropriate. Do not allow one abusive tenant to monopolise generation, export or publishing capacity. Fair-use limits should degrade expensive actions gracefully, not erase or lock out customer-owned content.

8. Website Configuration and Composition Engine

The composition engine builds a valid site plan from business goals, selected theme and the registered section catalogue. The engine is not a cross-site similarity detector. It relies on local constraints, weighted selection, stable randomness and human-tested design rules.

8.1 Canonical site configuration

The canonical site configuration is a versioned structured document. It describes metadata, theme tokens, page routes, section instances in order, content payload references, asset bindings, navigation, footer settings, features and export dependencies. The configuration does not embed untrusted arbitrary source code. It references registered component IDs and versions.

8.2 Composition algorithm

17. Classify business type and primary conversion goal (enquiry, call, visit, request quote, portfolio view, subscription or other supported goal).
18. Select a compatible global visual direction based on explicit user choices and safe defaults.
19. Choose the baseline sitemap; include mandatory core pages and add industry-relevant optional pages only when supported.
20. For each page, obtain candidate sections from the registry by page type, industry tags, component requirements, theme compatibility and feature flags.
21. Apply constraints: hero first; minimum/maximum section counts; avoid repeated visual patterns on the same page; require CTA/contact paths; respect media availability; do not duplicate the same section variant adjacent to itself.
22. Use weighted deterministic selection seeded by site ID plus composition version. Avoid costly global comparisons against all other sites.
23. Resolve content schemas and media slots for each selected section, then enqueue page generation jobs.
24. Validate the overall plan, required routes, section IDs/versions and layout rules before generation proceeds.
25. Persist plan version and selected component manifest for reproducibility, review and rollback.

8.3 Composition constraints

Constraint	Example rule
Page skeleton	Homepage includes a hero, business/context section, primary offer preview and a clear CTA; optional testimonial section requires genuine source material.
Variant diversity	Avoid adjacent sections that use the same structural pattern or duplicate the same visual rhythm.
Theme compatibility	Some display fonts, animation presets or layouts may be disabled for particular density/contrast combinations.

Constraint	Example rule
Content capacity	Select section variants with content fields capable of representing the actual number of services/testimonials/items.
Asset fit	A hero carousel requires enough appropriate wide assets; otherwise choose a supported single-image hero.
Accessibility	Exactly one meaningful H1 per page unless implementation rationale says otherwise; heading levels remain semantic; motion respects preference.
Mobile usability	Section layout must have a tested mobile template, not merely shrink desktop columns.
Determinism	Same site plan version and composition seed produce the same selection unless the user deliberately requests regeneration.

8.4 Incremental edits

Composition supports adding, removing, replacing and reordering a section as a change-set. The patch identifies page ID, section instance ID, intended operation and expected current version. The backend verifies that the target exists and the change preserves mandatory elements. The resulting draft config receives a new revision; prior published config is untouched until publish succeeds.

9. Section Registry and Manifest Contract

The registry is the contract between design-system engineers, the composition engine, AI prompts, the renderer and the export compiler. Adding a new section should not require rewriting prompt logic or database schemas for each field.

9.1 Registry design

Components live in version-controlled code. A manifest exports structured metadata that can be loaded by the application and validated in CI. The manifest specifies schema, rendering component, content instruction, allowed placement, tags, required feature support and animation presets. Runtime composition accepts only registered ID/version pairs.

9.2 Manifest requirements

Field	Requirement
id, version	Stable ID and semantic component version; never reuse an ID for a materially incompatible schema.
label, description	Human-readable description and composition semantics.
pageTypes, industries, goals	Compatibility filters for pages and business categories.
contentSchema	Typed fields, arrays, limits and optionality; schema is authoritative.
promptTemplateId	Versioned content-instruction reference; should align with schema version.
mediaSlots	Required/optional images, accepted aspect ratio, subject tags, count and focal rules.
layoutMetadata	Structural type, columns, density, responsive behaviour and known constraints.
themeCompatibility	Token and typography constraints.
animationPresets	Allowlisted animation identifiers only.
accessibilityContract	Semantic HTML, focus behaviour, alt-text expectations and reduced-motion plan.
exportCapabilities	Static-safe, client interaction required, external dependency required or unsupported.
testFixtures	Representative content, missing fields, long text, mobile snapshot and export fixture.

9.3 Component lifecycle

26. Draft: authoring only; unavailable to composition.
27. In review: unit, visual, accessibility, performance and export tests run.
28. Released: approved manifest and implementation can be chosen for new site versions.
29. Deprecated: excluded from new compositions; existing sites continue to render using compatible code or a migration path.
30. Retired: only after confirmed migration/rollback and retention window; never silently delete a version referenced by published sites.

9.4 Content lengths and item counts

Each field defines minimum/maximum words or characters only where useful; enforce array counts and character limits mechanically. Word-count instructions are guidance to the model, while runtime validators remain authoritative. The UI should detect overflow and offer a targeted rewrite or safe truncation only where the field allows it. Do not silently cut important legal copy or user-authored content.

10. Theme, Typography and Design Tokens

Theme tokens create variation without allowing combinations that compromise readability. Theme selection influences page appearance but must never alter the meaning or ownership of business data.

10.1 Token categories

Token group	Examples	Rule
Colour	primary, secondary, accent, background, surface, text, muted, border, success, error	Contrast checked on intended foreground/background combinations.
Typography	display family, body family, sizes, line heights, weights, letter spacing	Only bundled/approved fonts; avoid remote third-party font dependency where export can include assets.
Spacing	section padding, container gap, grid gap, card padding	Scale-based values rather than arbitrary values per section.
Shape	card radius, button radius, border style, shadow levels	Controlled variants; keep overall theme coherent.
Layout	container width, reading width, grid density, image ratio	Responsive tokens per breakpoint.
Motion	duration, easing, distance and animation preset	Motion respects reduced-motion and performance budgets.
Background	solid, subtle gradient, approved texture or image	No unreadable overlays; keep high-resolution background loading controlled.

10.2 Theme generation

The user may provide brand colours/logo or select a broad style. If brand assets are provided, prefer those colours while deriving accessible supporting tokens. If no preferences are supplied, infer a safe default from industry and stated tone—not ethnicity or an assumed demographic. A theme is a supported token set plus fonts and compatible layout variants; the model can recommend a theme ID and token adjustments within limits, but cannot inject arbitrary CSS.

10.3 Theme validation

- Contrast test text, button, link, form and focus-state combinations.
- Check long headings, translated-like lengths and narrow mobile widths.
- Verify sticky header, footer columns and floating CTA do not obscure core content.
- Check light/dark sections remain consistent with selected tokens.
- Snapshot-test theme variants across representative pages and breakpoints.
- Provide a fallback when a font or remote asset fails to load.

11. Multi-Stage AI Orchestration

The backend owns generation stages, dependencies, retries, usage accounting and schema enforcement. A model response is a proposal, never authority to publish, charge money, buy a domain or execute code.

11.1 Pipeline stages

Stage	Input	Output	Dependency / notes
A. Intake normalisation	User answers, uploaded business material, account/site context	Validated BusinessBrief JSON	Deduplicate facts, record source and confidence; missing essential facts create follow-up questions.
B. Site strategy	BusinessBrief, industry rules, available page and section catalogue summaries	SitePlan JSON with routes/theme/section IDs	Validate route list and registry IDs; cannot add unsupported features.
C. Content plan	SitePlan, section schemas, required page content matrix	Generation job plan	Backend creates jobs and prompt versions; model does not schedule provider jobs itself.
D. Page copy	BusinessBrief, page goal, selected manifests, approved related page facts	PageContent JSON per page/section	Can run in parallel for independent pages; enforce no invented claims.
E. Launch blog drafts	BusinessBrief, topical plan, internal routes, editorial guardrails	Separate ArticleDraft JSON per article	Generate each article as its own job; default to draft/unpublished state.
F. Asset resolution	Manifest media slots, context, representation preferences, asset catalogue	AssetBindings JSON / DB rows	Deterministic server-side selection from approved assets; fallback if insufficient assets.
G. Validation/repair	Structured output and schema errors	Validated content or targeted repair job	One bounded repair loop by default; do not blindly retry all stages.
H. Preview build	Validated config/content/theme/assets	Preview build ID, checks and URL	Run route/link/media/accessibility/performance checks.
I. Publish	Customer approval, selected plan, payment/domain preconditions	Immutable publish release + routing update	Side effects authorised by backend; maintain rollback target.

11.2 Dependency graph and concurrency

```
Intake -> Site Plan -> (Home Content || About Content || Services Content
                        || Contact Content || Blog Article 1
                        || Blog Article 2 || Blog Article 3)
       -> schema validation / bounded repair -> asset binding
       -> preview build -> customer review -> payment / publish preconditions
       -> publish release -> domain readiness checks
```

Independent page jobs may run concurrently to reduce total time, subject to provider concurrency limits, per-tenant limits and cost budgets. Jobs that depend on facts from the plan wait for it. Site plan, business brief and selected manifest versions are stored as immutable job inputs so a retry does not silently use a different design library version.

11.3 Job state model

State	Meaning	Allowed transitions
queued	Persisted and ready for a worker	running, cancelled
running	A worker owns a lease	succeeded, retry_wait, failed, cancelled
retry_wait	Temporary failure with scheduled retry	running, failed, cancelled
succeeded	Output persisted and validated	No re-run except explicit regeneration creates a new job
failed	Retries exhausted or permanent error	manual_retry creates a new attempt/job
cancelled	User/admin cancellation accepted	New build needed to resume

State	Meaning	Allowed transitions
awaiting_review	Preview generated and requires owner approval	approved, changes_requested, expired
published	Release pointer activated	Superseded by a later release; rollback creates/activates another release

11.4 Usage accounting

Record provider, model, input/output tokens where available, elapsed duration, job type, prompt version, retry count, site ID, outcome and cost calculation basis. Prices change, so store metered usage separately from price-book snapshots. Every chargeable action has a stable idempotency key. A retry of the same job should not produce duplicate billing ledger entries, while genuine new regenerations are separate operations.

11.5 Failure handling

- Retry transient timeouts, rate limits and selected 5xx failures with bounded exponential backoff and jitter. Honour provider retry hints where available.
- Do not retry deterministic schema/business-rule failures indefinitely; generate a targeted repair prompt or fail with an actionable message.
- Persist each completed step output so a downstream failure does not force successful expensive stages to rerun.
- Use per-provider concurrency caps and circuit breakers for widespread failures.
- Expose plain-language progress and errors to the customer; log technical details separately with secrets and sensitive content redacted.
- If a user cancels a build, stop downstream jobs when safe and record usage already incurred.

12. Prompt Contracts and Content Quality

Prompts are versioned product assets. Their outputs are constrained by the business brief, page objective, manifest fields, fact policy and schema. Prompt engineering is tested as software, not edited ad hoc in production.

12.1 Prompt contract composition

A runtime prompt should be assembled from separately versioned components: system-level guardrails, stage objective, business brief, selected component manifest, page context, relevant approved site facts, style/tone rules, field-length rules, schema, and prohibited behaviours. Persist prompt version identifiers and the input-data version rather than unnecessarily duplicating every secret or personal detail in logs.

12.2 Factuality policy

- Never invent awards, testimonials, client logos, customer counts, certification, professional credentials, prices, guarantees or years in business.
- Use supplied facts as source data; when material information is absent, draft neutral wording or mark a field as needing owner input.
- Testimonials are only populated from owner-supplied text or an approved, clearly identified source; otherwise the section is omitted or replaced.
- Do not fabricate physical addresses, opening hours, maps or business registration claims.
- Represent unknowns as missing/needs_input, not made-up content.
- Keep legal, health, financial and regulated-industry claims under stricter templates and owner review; do not imply professional advice.

12.3 Schema and content validation

Check	Technique	Failure response
JSON structure	Provider schema mode plus Zod parse	Bounded repair, then show job error
Required fields / counts	Schema and registry manifest	Repair only missing/invalid fields if possible
Length range	Word/character counter	Targeted rewrite; warnings for slight guidance drift

Check	Technique	Failure response
URL fields	URL parser + allowlist/policy	Reject javascript/data and malformed URL schemes
HTML/content injection	Render plain text or sanitized rich-text AST; never trust model-produced HTML	Sanitize/reject unsafe markup
Unsupported facts	Compare claim categories to business brief/source references and review rules	Remove claim or ask customer for evidence
Duplicate copy	Compare within current site/page and prompt constraints	Rewrite or suggest a different section; no cross-site comparison
Editorial quality	Rubric eval + sample-based human review	Flag for review; do not block all builds for subjective style variance
Prompt injection	Treat customer/file/web text as data; quote or delimit facts	Ignore requests to change system rules, expose secrets or perform side effects

12.4 Prompt registry and rollout

Store prompt template ID, semantic version, stage, output schema version, model eligibility, feature flags and evaluation dataset version. Test a new prompt on a fixed benchmark of representative industries and adversarial briefs. Run canary generations before promoting; monitor cost, repair rate, claim violations, edit success and human quality scores. Keep rollback to the previous prompt version.

13. Conversation and Incremental Editing

The chat is the primary control surface, not a universal free-form command channel. The backend classifies a customer's request into supported edit operations and converts it to an explicit change set that can be previewed and approved.

13.1 Conversation context

Conversation state includes account/site ID, current draft version, business brief reference, selected page/section context, recent operations and relevant site facts. Store conversation messages under access control and apply a retention policy. Avoid sending an entire history on every call; summarise or retrieve only the relevant facts and sections needed for the operation.

13.2 Intent categories

Intent	Operation	Guardrails
Content edit	Patch text field or content block	Do not regenerate unrelated sections; preserve approved facts.
Visual adjustment	Update approved theme tokens or select supported variant	Validate contrast and compatibility; no arbitrary CSS.
Image change	Change asset binding	Use curated/approved assets only; validate licence and section fit.
Section insertion	Add a registered section to an allowed slot	Check count, ordering, schema and CTA requirements.
Section reorder/remove	Change site config order or remove optional block	Protect mandatory contact/navigation functions; confirm destructive changes.
Page creation	Add route from approved page template	Check plan allowance, navigation and canonical URL impact.
Blog article	Generate a new ArticleDraft job	Respect allowance and publish status; verify internal links.
Domain/billing action	Guide into explicit confirmation UI	AI cannot directly register, cancel or pay without authenticated backend confirmation.
Unsupported function	Explain current product boundary and offer allowed alternative	Never fake functionality or promise unavailable integration.

13.3 Preview/approval policy

Changes affecting many pages, theme tokens globally, route slugs, domains, billing, removal of existing content or search indexing should require an explicit confirmation step. Minor text edits may be offered for rapid preview, but the live published version remains unchanged until publish or an explicitly enabled auto-publish rule succeeds.

13.4 Conversation safety

The user's request, uploaded files and any imported content are untrusted data. The model must not be able to choose arbitrary tools, write production code, modify system policy, retrieve other tenants' data, reveal provider keys or issue payments/domain registration actions. The server maps approved intents to a fixed capability set and independently enforces authorisation.

14. Cloudinary Asset Catalogue and Selection

Cloudinary stores and transforms approved image assets; PostgreSQL stores the application catalogue, relationships, approval state, licensing metadata and section-selection tags. Avoid using unverified third-party stock page URLs in live sites.

14.1 Asset ingestion pipeline

31. Curator uploads original asset to controlled Cloudinary folder or upload flow.
32. Ingestion checks file type, size, dimensions, animation policy, malware/scanning requirements and metadata completeness.
33. Create/update asset record with stable Cloudinary public ID, source, licence, expiry/attribution requirements, subject, visual style, suitable pages/sections, orientation and focal metadata.
34. Generate approved responsive transformations and thumbnails. Retain original or backup according to rights and storage policy.
35. Human review approves the asset for production use; licence-sensitive assets may require explicit expiration/review date.
36. Asset is available for matching only when `status=approved` and rights have not expired or been revoked.

14.2 Suggested asset schema

Field	Type / intent
asset_id	Internal UUID
cloudinary_public_id	Stable media provider identifier
status	draft, review, approved, suspended, expired, deleted
source_provider / source_url	Provenance where permissible
licence_type / licence_reference	Rights and evidence
licence_expiry_at / attribution_text	Expiry and required credit if any
industry_tags / subject_tags	Tourism, restaurant, professional team, workspace, etc.
section_tags / use_cases	Hero, gallery, service card, blog feature, avatar, background
region_context / environment_tags	Kenya, East Africa, urban, coast, studio, landscape; avoid overclaiming identity
representation_tags	Explicit/approved representation metadata where relevant and justified
aspect_ratio / width / height	Technical match
palette_tags / style_tags	Colour and visual direction
focal_x / focal_y	Crop focal point
alt_text_default / caption	Accessible description and optional caption
reviewer_id / reviewed_at	Governance and provenance

14.3 Selection algorithm

Given a component media slot, filter to approved and unexpired assets whose licence permits the intended use. Rank candidates by compatible use-case, subject, aspect ratio, theme/style, geographic context and user-provided representation preferences. Apply diversity

within the current site so the same image is not repeated needlessly. Choose a deterministic shortlist using a site/section seed, then persist the selected asset IDs. If the shortlist is inadequate, use a safe neutral fallback or omit an optional slot; do not invent a URL.

14.4 Cloudinary reliability and exports

Use named transformations or controlled URL generation so dimensions and quality are repeatable. Avoid requiring unrestricted Cloudinary credentials in the browser. For static export, download the original/appropriately transformed image into the export bundle if licence permits and the customer selects a self-contained export. Otherwise retain a remote asset URL and flag it as an external dependency. Export documentation must identify assets that cannot legally or technically be transferred.

15. Media Representation and Regional Context

Imagery should feel relevant and respectful. Regional context can guide scenery, language, architecture, products, work settings and cultural details. The platform must not infer an individual's ethnicity from location or use a rigid continent-to-race mapping.

15.1 Supported preference model

Optional preferences may cover audience market, language, representation style, inclusive/diverse imagery, relevant local context, image mood and subjects to avoid. Prefer explicit owner intent and context tags over sensitive demographic inference. Store only preferences needed for asset selection and allow the customer to review the chosen imagery.

15.2 Representation guidelines

- A Kenyan local-service website may prefer appropriate Kenyan/East African context when the image reflects the actual business and audience.
- A global business based in Kenya can choose broadly international or diverse imagery instead.
- Do not assume that all people in a geographic region share an ethnicity, appearance, language or preference.
- Do not tag a person's ethnicity without an appropriate, lawful basis and reliable metadata source; generic image licensing often does not support such claims.
- Never describe the represented person as the business owner, staff member or customer unless the business owner has confirmed that fact.
- Avoid images that perpetuate stereotypes or misrepresent products, facilities or services actually offered.

15.3 Image-quality checks

Run a manual review of high-visibility hero and team images in the starter catalogue. Automated checks should reject broken/missing assets, unsuitable aspect ratios, low resolution and disallowed licence states. Visual-quality review remains necessary for contextual relevance and representation quality.

16. Site Pages and Standard Components

The initial product uses a standard multipage structure with industry-appropriate additions. Each page is a route composed of registered sections, and the global shell owns navigation, footer, cookie/privacy links if needed, and shared floating actions.

16.1 Standard page requirements

Page	Baseline content	Rules
Home	Hero, About preview, services/products preview, why choose us/benefits, testimonials if genuine, primary CTA, optional FAQ/portfolio/stats	Hero and CTA must express a clear business value proposition; optional sections depend on evidence and industry.
About	Company/person story, mission/approach, team or values where supplied, CTA	Do not invent founding date, staff or credentials.
Services/Products	Service or product groups, descriptions, details/CTA, optional pricing if supplied	No fake prices or unsupported service guarantees.
Blog	Article index, categories/tags if available, article pages, pagination	Articles are drafted, reviewed and published with unique slugs and metadata.
Contact	Supplied contact details, contact form, service area/location if provided, WhatsApp/call/email actions	Validate inputs, protect from spam, disclose submission purpose.

Page	Baseline content	Rules
Privacy/Terms	Platform/site owner-provided policy and generated template with reviewed variables	Avoid pretending a generated template is legal advice; require owner approval and jurisdiction selection where appropriate.

16.2 Header and navigation

Header is sticky where supported by the selected layout, but it must not consume excessive mobile viewport height. Desktop navigation exposes the primary page routes; mobile navigation becomes a tested menu with keyboard focus handling and accessible expanded state. The active route is represented semantically, and all navigation remains usable without animation.

16.3 Four-column footer

Default desktop footer is a four-column design that can support brand summary/logo/social links, primary navigation, key services/resources and contact/newsletter details. Columns collapse to two or one column based on breakpoint and content. The footer should not render fake address/phone/hours. Newsletter capture appears only when the business owner has configured a lawful provider or the platform's supported form pathway.

16.4 WhatsApp and CTA controls

The floating WhatsApp button appears only if a valid business WhatsApp destination is supplied and validated. It must not obscure cookie controls, chat widgets, form submit buttons or mobile browser safe areas. Use approved URL construction with encoded initial text; do not accept unsafe schemes. Main CTA buttons should have a defined purpose and destination.

16.5 Testimonials and social proof

EVIDENCE REQUIREMENT Do not generate fictional customer testimonials, ratings, certifications or named endorsements. If the customer has no approved testimonials, choose another section such as process, benefits, gallery or portfolio.

17. Blog, Content Management and SEO

Blogging is a core website feature and contributes to customer value, but content must be accurate and editable. Technical SEO is part of generation and publishing; ranking outcomes are not guaranteed.

17.1 Blog data model

Each article has title, slug, excerpt, body representation, author/display name, category/tags, created/updated dates, publication state, featured asset, canonical URL, SEO title/description, internal links, editorial status and generation provenance. Initial launch articles should be drafts pending review. Avoid auto-publishing high-volume content without owner permission.

17.2 SEO generation

- Unique page title and meta description based on page intent and supplied business context.
- Canonical URL consistent with production hostname and route, updated after domain changes.
- A sitemap includes only public canonical pages intended for indexing; previews and drafts are excluded.
- Robots configuration blocks internal dashboards, preview builds and private routes while allowing intended public pages.
- Open Graph/social metadata and suitable image references per page.
- Semantic page headings, crawlable internal links and valid HTTP status codes.
- Structured data only when the page genuinely meets the relevant schema, using verified facts and no fabricated ratings/reviews.
- Image alt text describes relevant visual content rather than stuffing keywords.
- Redirect rules preserve previous slugs where possible when page URLs change.
- No promise of indexation, ranking position, traffic or SEO result.

17.3 Search Console and analytics

Later, provide an explicit Google authorisation flow for connecting Search Console and supported analytics. Request the minimum necessary scopes; allow disconnection and explain which data is retained. Keep integration tokens server-side and encrypt sensitive credentials. Do not make Search Console connection a prerequisite for publishing a valid sitemap.

18. GSAP Animation System

GSAP is an enhancement layer for selected section experiences. It must not be required for the content to render or for primary navigation/CTAs to function.

18.1 Animation preset catalogue

Preset	Use	Constraints
fade-up	Small group/section entry	Respect reduced-motion, avoid large travel distance.
stagger-cards	Service/process cards	Limit stagger duration; do not delay essential content excessively.
image-reveal	Selected editorial image	Ensure no hidden content when JS fails.
hero-crossfade	Supported hero imagery	Preload first image, avoid excessive simultaneous large assets.
subtle-parallax	Optional background/visual detail	Disabled on reduced motion and low-capability paths where performance tests require.
count-up	Verified numerical fact only	Only animate confirmed numeric data; provide static accessible final value.
micro-interaction	Button/menu feedback	No animation-only communication; focus state always visible.

18.2 Implementation rules

- Manifests reference allowlisted preset IDs, never generated JavaScript snippets.
- Animation initialization occurs only when client-side hydration is ready and must be cleaned up on React unmount.
- Avoid animating layout-critical properties that cause frequent layout/reflow; prefer transform and opacity when possible.
- Honour `prefers-reduced-motion`; the reduced state should display the same information immediately and coherently.
- Test SSR/hydration and route changes to avoid duplicate timelines or memory leaks.
- Lazy-load below-the-fold animation code only if doing so does not hide essential content or create layout shifts.
- Measure runtime JS and Core Web Vitals with animation on and off.

19. Responsive and Accessible Design

Both the customer dashboard and generated websites are mobile-first. “Responsive” means each layout is designed intentionally for narrow screens, not just scaled down from desktop.

19.1 Breakpoints and layouts

Define a tested breakpoint scale, for example small phone, large phone/tablet, laptop and wide desktop. Components must remain fluid where practical. The exact pixel cutoffs are design-system constants, not scattered per component. Test common widths, zoom to 200%, mobile landscape and touch target sizes.

19.2 Dashboard navigation

Viewport	Navigation	Notification placement
Desktop/laptop	Persistent or collapsible left sidebar: Home, Build, Websites, Billing as needed, Profile/Settings	Top-right bell with unread state and dropdown/page.
Mobile	Bottom navigation: Home, Build, Websites, Profile	Top-bar bell; long settings lists scroll within page, never hide below nav.
Tablet	Responsive sidebar or compact navigation depending on width	Top bar remains available.

Viewport	Navigation	Notification placement
Accessibility mode	All destinations reachable with keyboard/screen reader; visible focus	Announcements for unread counts and success/failure states are accessible.

19.3 Accessibility requirements

- Target WCAG 2.2 AA for core dashboard and generated component library, with a documented exception log if a requirement cannot initially be met.
- Use semantic landmarks, headings, labels, error messages and accessible names.
- Keyboard-operable menus, dialog focus management and escape/return focus behaviour.
- Colour contrast checks and non-colour-only status indicators.
- Alt text controls tied to curated assets and user edits; decorative images have empty alt where appropriate.
- Reduced motion respected across GSAP and CSS transitions.
- Forms provide accessible inline errors and not only toast notifications.
- Test screen reader flows for signup, site creation, preview approval, payment status and export download.

20. Hosted Publishing and Routing

Hosted publication uses the shared renderer and a stable published-release pointer. A user can preview a draft independently from the currently live version. The live site changes only after checks and an authorised publish operation succeed.

20.1 Release model

Every publish creates an immutable Release record containing the site config version, content version, theme version, selected component versions, asset bindings, build output/check results and publishing actor/time. A site-level `publishedReleaseId` pointer identifies the version served to public traffic. Publication switches that pointer atomically after verification; rollback points it to a prior valid release or creates an explicit rollback release.

20.2 Publish sequence

37. Verify the acting user owns the site and the plan/site is eligible to publish.
38. Validate all routes, section IDs and versions, required page content, assets and feature dependencies.
39. Build or pre-render public page data and run critical checks.
40. Confirm custom domain is verified and service plan covers required feature; for platform subdomain, verify that it is reserved and unique.
41. Write immutable release and build artefact metadata.
42. Atomically move public publish pointer, invalidate/rotate caches and ensure domain routing resolves to the same site.
43. Run smoke tests against platform URL and custom domain; monitor first requests.
44. Notify the owner and record audit event. If checks fail, keep the previous release live.

20.3 Preview hosting

Preview URLs should be unguessable or protected by signed access, time-limited where feasible, and blocked from indexing. Preview content should not be served from a tenant's public canonical domain. Preview state is separate from the public release pointer, and preview URLs are invalidated when no longer required or when access is revoked.

20.4 Cache invalidation

Cache keys include canonical site ID, release ID and route. Avoid keying solely by Host header without normalisation. On publish, new release IDs naturally avoid stale content; invalidate associated metadata and public hostname resolution caches. Domain removal or ownership change requires immediate cache purge and a safe fallback route.

21. Static Export and Portability

Export gives customers a practical route to independent hosting. It uses the canonical site configuration and approved content/assets, not a scrape of the currently rendered website.

21.1 Export tiers

Tier	Contents	Target user	Scope
Portable static package (MVP)	Per-route HTML, CSS, JavaScript for supported interactions, local/permitted images, sitemap/robots where configured, README, dependency report	Business owner using ordinary static hosting	Must not require local build tools.
Connected deployment (later)	Repository or provider integration, environment mapping, deploy status, supported CI configuration	Technically comfortable owner/agency	Requires token/OAuth permission and deployment adapter.
Developer source package (later)	React/TypeScript source, manifests, theme/config source, lockfile and build scripts	Developer/agency	Clearly list runtime services and minimum supported Node/tool versions.

21.2 Export build steps

45. Freeze export input: config version, content versions, asset bindings, section registry versions and export compiler version.
46. Run export compatibility analysis for every section and configured feature.
47. Convert each route to static HTML plus required CSS/JS and local asset references.
48. Resolve canonical URLs and generate a sitemap limited to exported public routes.
49. Rewrite or preserve asset URLs according to licence and customer option; verify all required assets are present.
50. Identify forms, newsletter integrations, analytics, search, live data and APIs that depend on backend services.
51. Generate dependency report with each supported, replaced, external, unsupported or customer-configured item.
52. Package output with a README, deployment notes, license/attribution list and known limitations.
53. Run local static server smoke tests across all routes, links, forms fallback and responsive assets.
54. Provide an export checksum, manifest/version and secure download URL with expiry.

21.3 Static-export constraints

Not every Next.js capability is available in pure static export. Server actions, request-time server logic, databases, authentication, dynamic image optimisation and APIs require alternative implementations or external services. The export validator must fail or warn based on explicit severity rather than pretending backend features still work. For example, a contact form can use an external form service selected by the owner, be delivered through a supported platform API while hosted with the platform, or export as a mailto/contact alternative if acceptable; the user must choose knowingly.

21.4 Export versioning

Keep a record of each generated package tied to exact input versions. A future edit generates a new package; do not mutate an existing downloaded artifact. For customers using connected deployment later, the platform should provide a release diff, deployment logs and the ability to disconnect. A downloaded static site does not update on its own.

21.5 Export safety

- Never execute arbitrary code supplied by a customer or AI in the app server process.
- If a developer source build is supported, build in an isolated sandbox with strict CPU, memory, process, filesystem and network limits.
- Exclude secrets, private prompts, other tenant data, database credentials and provider keys from exports.
- Do not bundle paid/licensed images unless export rights allow redistribution; report excluded assets and supply approved substitutes.
- Scan exported files for accidental secrets and internal URLs.

22. Domains, DNS, TLS and Registrar Adapter

Domain management is a durable lifecycle workflow, not a single button. Registrar registration, DNS mapping, HTTPS readiness and website publication are distinct states and must be displayed separately.

22.1 Domain states

State	Description	Transition conditions

State	Description	Transition conditions
searched	Availability query performed	Fresh provider response; availability is time-sensitive.
reserved_in_checkout	Customer has selected the domain while checkout runs	Short-lived internal reservation only; not a registrar guarantee.
payment_pending	Payment initiated	Server-side payment status pending.
registration_pending	Payment confirmed and registrar call queued	Idempotency key and validated registrant/contact data present.
registered	Registrar confirms registration	Persist provider domain ID, dates and registrant receipt/records.
dns_pending	Nameserver/record setup not verified	Provider state may lag; retries with timeout.
verification_pending	Custom hostname challenge or ownership proof pending	Domain owner/control challenge issued.
tls_pending	Hostname is routed; certificate not yet ready	Provider reports issuance in progress.
active	DNS/hostname, TLS and site mapping verified	End-to-end health check passes.
renewal_due	Renewal window reached	Reminder policy executes.
renewal_failed	Registrar renewal attempt failed	Recovery workflow and alert; site status remains distinct from domain status.
expired / transfer_pending / cancelled	Terminal or special lifecycle states	Customer guidance and transfer/expiry policy required.

22.2 Registrar adapter interface

```
interface RegistrarAdapter {
  checkAvailability(input: DomainSearchInput): Promise<AvailabilityResult>;
  quoteRegistration(input: DomainQuoteInput): Promise<DomainQuote>;
  registerDomain(input: RegistrationRequest, idempotencyKey: string): Promise<RegistrationResult>;
  getDomain(domain: string): Promise<RegistrarDomainStatus>;
  renewDomain(input: RenewalRequest, idempotencyKey: string): Promise<RenewalResult>;
  transferOut?(input: TransferRequest): Promise<TransferResult>;
  updateRegistrant?(input: RegistrantUpdate): Promise<RegistrantUpdateResult>;
}
```

22.3 Pricing and ownership

- Show first-term price, applicable taxes/fees, renewal price, renewal period and cancellation/transfer information before checkout.
- Register using customer registrant details or legally appropriate process; do not make the domain an undisclosed MegaScript-owned asset.
- Verify contact email/eligibility where required for the TLD.
- Persist registrar response and customer confirmation/receipt.
- Keep domain billing separate from hosting so customers can understand renewal obligations.
- Provide clear route to transfer a domain away and avoid dark patterns or unnecessary transfer obstruction.
- Confirm actual registrar API support and policy for each TLD, especially `.co.ke`, before advertising automation.

22.4 DNS and TLS operations

For platform-managed nameservers, create only records required by the verified custom-hostname workflow. If customer-owned DNS is used, show exact record instructions, poll records safely and validate hostname ownership. Never claim active until the host provider confirms status and a real HTTPS check passes. DNS propagation can be variable; show progress and last-checked time rather than a false time guarantee.

22.5 Domain security

Prevent domain takeover by requiring a supported ownership challenge before activation; remove mapping immediately on successful transfer/removal. Treat stale mappings as a high-priority security risk. Never allow arbitrary external redirect or CNAME assignment to select another tenant. Log changes to registrant details, nameservers, domain attachment and transfer requests.

23. Authentication, Sessions and Verification

Authentication should be provided by a well-maintained library/service rather than a bespoke session implementation. The frontend communicates account state; the backend enforces every protected action.

23.1 Supported entry methods

- Email and password, with verified email required before publishing or domain purchases.
- Continue with Google OAuth, with safe account-linking and verified provider email checks.
- Optional SMS/mobile verification added only after threat/risk and cost review; do not make it mandatory by default unless business policy requires it.
- Password reset and email change with time-limited, single-use verification tokens.
- Future MFA for platform administrators and possibly customer accounts at elevated risk.

23.2 Session requirements

Control	Expected behaviour
Session cookie	Secure, HttpOnly and appropriate SameSite settings; no session ID in URLs.
Session rotation	Rotate after authentication, privilege change and material account security events.
Expiration	Idle and absolute expiry appropriate to user risk; sensitive operations may require recent authentication.
Logout	Invalidate server-side session where supported and clear client cookie.
Account link	Require proof of both identities or a documented safe provider flow; prevent takeover via email mismatch.
Email verification	Time-limited one-time token/code; resend throttling and invalidate old tokens as appropriate.
Password storage	Delegate to vetted auth provider/library using current password-hashing practices; never log passwords or verification secrets.
Recovery	Recovery flows should not disclose whether arbitrary emails have accounts beyond necessary UX and should rate-limit attempts.
Admin security	Require stronger protections, minimal privileges and audit events for critical actions.

23.3 Authorisation

Use server-side role checks plus resource ownership checks. A user having a valid session does not imply they own a supplied `siteId`. Every site mutation checks ownership and plan entitlements; every admin action checks a dedicated permission. Access tests should attempt cross-tenant reads and writes for every major API family.

23.4 Account lifecycle

Support verified-email update, password reset, account deactivation, data export and deletion requests, and account recovery. If an account has an active registered domain or paid term, explain implications and options before destructive actions. Preserve legally required transaction records according to the defined retention policy, while deleting or anonymising personal data where required.

24. Billing, Plans, Usage and Payments

Billing is subscription-centred, with a customer-friendly monthly or annual fee and a bounded AI allowance. Domain registration and renewal are separately itemised because they have distinct prices and lifecycle obligations.

24.1 Plan model

Plan attribute	Configurable field / enforcement

Plan attribute	Configurable field / enforcement
billing interval	monthly or annual
site allowance	number of sites in account
page allowance	number of pages or feature-based quota
AI editing allowance	credit/operation budget per billing period; map internally to measured cost
blog generation allowance	number of article jobs per period
export entitlement	portable static export / developer source export
hosting limits	traffic, storage, media and supported feature levels
custom-domain entitlement	number of connected domains and included hostname workflow
support tier	response channel and support scope
retention/grace period	behaviour after failed payment or cancellation

24.2 Checkout sequence

55. Create an order with immutable line items for plan term, domain registration, taxes/fees and any one-time setup charges.
56. Create payment intent/session through server-side provider adapter; do not trust price or amount submitted by browser.
57. Customer completes supported payment method.
58. Receive verified server-side provider event; verify signature or authenticated retrieval and deduplicate by provider event/payment ID.
59. Update payment state and ledger within a transaction; enqueue fulfillment using a unique idempotency key.
60. Fulfill subscription/domain steps in a durable workflow; if provider confirms charge but fulfillment fails, retry without charging again and alert operations if unresolved.
61. Send receipt and update dashboard only after trusted status is recorded.

24.3 AI allowance semantics

Define whether allowance units represent generated page, article, edit operation or weighted credits. Recommended MVP: customer sees an understandable allowance such as AI edits/generation credits; internally, each job records actual token/provider cost. High-cost operations such as a complete new site or batch of article drafts consume more allowance than a simple field edit. State how failed provider jobs are treated; failures caused by the platform should not silently consume customer allowance. Abuse and repeated user-requested rewrites require fair-use limits.

24.4 Subscription state

State	Expected product behaviour
trial / active	Features available according to current plan and usage balance.
past_due	Grace period and notices; do not immediately destroy or export-lock customer content.
cancel_at_period_end	Keep features until paid period ends, with clear date shown.
cancelled/expired	Switch hosting/AI services according to disclosed retention and grace policy; preserve account/invoices and domain information.
payment_failed	Retry according to policy, notify, support payment update and log webhook state.
refunded/chargeback	Reconcile financial ledger, investigate abuse and handle content/domain rights separately.
domain_renewal_failed	Notify customer and distinguish domain expiry risk from hosting subscription status.

24.5 Payment-provider validation

IntaSend is a candidate for M-Pesa and card collection in the Kenyan market. Do not claim that all payment methods support recurring billing identically. Confirm the currently supported subscription method, webhook event types, refunds, fees, settlement timing, account requirements and terms directly with the provider before plan design is final.

25. Email and Notification Architecture

Brevo is the candidate for transactional email and consent-based platform newsletters. In-app notifications remain part of the application data model and should not depend on email delivery succeeding.

25.1 Message categories

Category	Examples	Consent / control
Account transaction	Verification, reset, address change/security notice	Necessary for account operation; security and legal policy applies.
Operational transaction	Build complete/failure, publish complete, domain ready, receipt	User-configurable preferences where compatible with safety/transactional needs.
Lifecycle/marketing	Welcome series, product news, feature announcements, promotional digest	Explicit opt-in where required; unsubscribe and suppression support.
Domain lifecycle	Renewal reminder, registrant verification, expiry risk	Time-based job with retry; domain notices must be accurate.
In-app notification	Unread alert, billing status, admin notice	Persist status, user preferences and read timestamp.

25.2 Deliverability and template rules

- Use verified sending domains and configure SPF, DKIM and DMARC according to the sender provider guidance.
- Use consistent From/Reply-To identity and branded but light HTML templates.
- Provide plain-text versions for key messages.
- Do not include passwords, long-lived authentication tokens or unnecessary personal data in email.
- Use expiring single-use verification links/codes and rate-limit issuance.
- Handle provider bounces and complaints; suppress repeatedly invalid destinations.
- Keep marketing contacts and consent records separate from transactional account events.
- Every marketing email has a working unsubscribe method and does not resubscribe a user without a valid action.

25.3 In-app notifications

Notification entity includes user ID, type, title/body, target route, createdAt, readAt, delivery status, channel and idempotency key. The bell shows unread count, but the UI should remain accessible without reading that count. Clicking a notification checks access to its destination; notification URLs must not expose tenant data to another user.

26. Database Design and Data Lifecycle

PostgreSQL is the primary system of record. Prisma supports typed schema and migrations; access controls must remain effective for any raw SQL, background job or administrative path.

26.1 Core entity map

```
User 1---N SiteMembership N---1 Site
Site 1---1 BusinessProfile
Site 1---N Page 1---N SectionInstance
Site 1---N SiteRevision 1---N Build / PublishRelease
Site 1---N DomainMapping; RegisteredDomain may link to CustomerAccount
Site 1---N Conversation 1---N Message
Site 1---N GenerationJob 1---N AIUsageEvent
Site 1---N AssetBinding N---1 Asset
User 1---N Subscription 1---N Invoice / PaymentEvent
User 1---N Notification / EmailPreference / ConsentRecord
```

```
SectionManifest + PromptTemplate + ThemeDefinition are global versioned registries
AuditEvent records sensitive administrative or lifecycle actions
```

26.2 Database conventions

- Use UUID or another non-guessable primary identifier consistently; do not expose sequential IDs where they invite enumeration.
- Include `created_at`, `updated_at`, and where relevant `created_by`, `deleted_at`, `version` and lifecycle status.
- Use foreign keys, unique constraints and check constraints for invariants that fit the database.
- Represent financial amounts as integer minor units plus explicit ISO currency, not floating point.
- Store provider identifiers and webhook idempotency identifiers with unique constraints.
- Use immutable version rows for published configs/releases and retain content snapshots needed for rollback.
- Treat hard deletion carefully; use soft deletion only where retention/security policy allows, then purge according to lifecycle policy.
- Avoid storing raw payment card information; let the PCI-compliant payment provider handle it.
- Use UTC timestamps in storage and present user-local date/time with explicit timezone rules.

26.3 Migrations

All schema changes go through reviewed, version-controlled Prisma migrations or a documented equivalent. Do not mix manual production SQL with untracked ORM schema changes. Use expand-migrate-contract for changes affecting large tables or mixed application versions. Test migration and rollback/recovery on a production-like database and include data backfill strategy when changing canonical configuration schemas.

26.4 RLS and application checks

If Supabase RLS is used, define policies for each tenant-owned table and test them with realistic JWT/session contexts. RLS policies and database grants are separate controls; both need validation. Prisma service-role access can bypass user-level row protections depending on connection/credentials, so service methods must still enforce ownership and admin scope. Never treat RLS alone as proof that application logic is correct.

27. API Design and Webhooks

APIs should have consistent naming, validation, error formats and idempotency conventions. Public browser requests never receive provider secrets or privileged database credentials.

27.1 API conventions

- Use resource-oriented route names and clear HTTP semantics, e.g. `/api/sites/{siteId}/revisions` and `/api/sites/{siteId}/generation-jobs`.
- Validate request bodies, query parameters, path IDs and outputs at runtime with versioned schemas.
- Return structured errors: code, user-safe message, request ID, retryable flag and field issues when appropriate.
- Enforce auth, ownership, plan entitlements and rate limits at the server boundary, not only in UI state.
- Use pagination/cursors for potentially large lists such as assets, articles, audit events and notifications.
- Use `Idempotency-Key` for chargeable or externally stateful actions where clients may retry.
- Add API contract tests and an OpenAPI document or comparable generated route reference.

27.2 Webhook handling

Webhook handlers should verify provider signatures or authenticate a provider retrieval call, record delivery ID and payload hash, reject invalid events, apply idempotency, process the smallest necessary state transition and enqueue long work durably. Return success only when the event is safely persisted for processing. If the provider expects quick acknowledgement, the handler should enqueue before returning and the worker should perform slow work. Redact secrets and sensitive payment data in logs.

27.3 Key API families

Family	Representative routes	Notes
Account	`/api/account`, `/api/account/email-verification`, `/api/account/preferences`	Session-protected; sensitive changes may require re-auth.
Sites	`/api/sites`, `/api/sites/{id}`, `/api/sites/{id}/brief`	Site ownership and plan limits enforced.

Family	Representative routes	Notes
Conversation	`/api/sites/{id}/conversations`, `/messages`	Persist user intent and associated generation operation.
Generation	`/api/sites/{id}/generation-jobs`, `/jobs/{jobId}`	Long-running tasks return job IDs; clients poll/subscribe.
Content revisions	`/api/sites/{id}/revisions`, `/pages/{pageId}`	Optimistic concurrency/version checks.
Preview/publish	`/api/sites/{id}/preview`, `/publish`, `/rollback`	Explicit approval and release creation.
Assets	`/api/assets/search`, `/api/sites/{id}/asset-bindings`	Only approved/licensed assets in production selection.
Domains	`/api/sites/{id}/domains`, `/api/domains/{id}/verify`	Registrar state and mapping state distinct.
Billing	`/api/billing/checkout`, `/portal`, `/usage`	Price is computed on server; webhook is authority.
Export	`/api/sites/{id}/exports`, `/exports/{id}/download`	Secure temporary download URLs; dependency report.
Notifications	`/api/notifications`, `/notifications/{id}/read`	User-bound records and idempotent creation.
Webhooks	`/api/webhooks/payments`, `/registrar`, `/email`	Provider-specific verification and event deduplication.

28. Background Jobs, Retries and Idempotency

Background jobs handle all work that can exceed a normal request lifetime or depend on third-party services. Durable execution must make retries safe and avoid re-running completed expensive work.

28.1 Job classes

Job class	Examples	Priority / throttle
Interactive high priority	Homepage draft, short edit, preview build	Per-user/site fairness; provider rate cap.
Build pipeline	Full site/page generation, asset binding, validation	Job-level budget and concurrency cap.
Publish pipeline	Release validation and routing switch	High reliability, idempotent release ID.
Domain lifecycle	Registration, DNS poll, TLS poll, renewals	External provider limits; retry with visible state.
Billing lifecycle	Webhook reconcile, invoice update, entitlement refresh	Exactly-once effect via idempotency/invariants.
Communication	Build finished, reminders, newsletter send	Rate limits and suppression policy.
Maintenance	Cleanup, backups, asset expiry, stale preview removal	Low-priority scheduled windows.

28.2 Retry policy

Retry only errors classified as transient; use exponential backoff with jitter and a maximum attempt/deadline. Provider rate-limit responses should respect retry hints. Permanent schema or eligibility failures transition to failed with a specific reason. Never retry a domain registration, payment charge or email campaign blindly without a stable idempotency key or reconciliation query.

28.3 Idempotency table

Operation	Idempotency strategy
AI stage	Unique stage job ID + input/config version; outputs linked to attempt and accepted output version.

Operation	Idempotency strategy
Payment webhook	Unique provider event ID and/or payment ID with processed timestamp.
Order fulfillment	Unique order/line-item fulfillment key; registration can only occur once for that purchased domain/term.
Domain registration	Registrar-supported request key if offered; otherwise pre/post status reconciliation before retry.
Publish	Unique release ID; repeated request returns the current result, not duplicate release side effects.
Email notification	Unique business event + recipient + template/version/channel key.
Export	Unique export job ID; retries produce same logical artifact or a new attempt tracked separately.
Renewal reminder	Domain ID + renewal date/window + reminder type.

28.4 Cancellation and compensation

Cancellation should prevent new downstream actions and stop safe active work. Completed external side effects may not be reversible; record them, notify the customer and use a compensating workflow where appropriate. For example, if payment succeeds but domain registration fails permanently, the system needs a visible recovery or refund path rather than silently leaving a paid order pending.

29. Security, Abuse Prevention and Secrets

The platform controls websites, domains, email, billing and AI. Security must be part of each workflow, not a final checklist.

29.1 Threat model

Threat	Impact	Controls
Cross-tenant data access	Expose/edit another site or customer data	Ownership checks, RLS where appropriate, resource-scoped service functions, adversarial tenant tests.
Domain takeover/misrouting	Attacker serves malicious content under a customer domain	Ownership verification, verified mapping state, strict hostname resolver and immediate detach/transfer controls.
Webhook forgery/replay	Fraudulent publishing, billing or domain state	Signature/authenticity validation, unique event IDs, idempotency and audit.
Prompt injection	Model reveals secrets or attempts unauthorized actions	Untrusted content boundaries; fixed tool allowlist; server-side authorization; no secrets in prompts.
XSS/HTML injection	Script execution in editor or customer site	Plain-text fields or sanitised structured rich text; CSP; output encoding and tests.
CSRF/session theft	Account takeover or unintended actions	Secure cookies, CSRF protections appropriate to framework, origin checks and session rotation.
Secret leakage through export/logs	Third-party accounts compromised	Secret scanning, redaction, allowlisted export files, separate credentials per environment.
Billing/domain abuse	Fraudulent registrations and high provider costs	Rate limits, verification, idempotency, transaction limits and abuse monitoring.
Malicious uploads	Content execution or storage abuse	MIME/signature verification, size quotas, processing isolation, scanning and image-only pipeline where possible.
DoS/cost bombing	AI/media provider costs spike	Per-account quotas, concurrency limits, spend alerts, rate limits and circuit breakers.

29.2 Security headers and browser controls

Set appropriate Content Security Policy, HSTS after HTTPS readiness, frame restrictions, MIME sniffing protection, Referrer Policy and Permissions Policy. Use restrictive script sources where feasible; document Cloudinary, analytics and payment-host requirements. Customer websites should inherit secure defaults but allow approved external integrations only through explicit configuration.

29.3 Secrets management

- Store credentials in environment-specific secret management, not source code or database records readable by customers.
- Separate development, staging and production keys.
- Use least-privilege provider tokens and rotate credentials with documented playbooks.
- Never put API keys in client bundle, prompt output, source export, logs or error messages.
- Use short-lived/signed upload operations when client-side upload is needed.
- Restrict egress from any build sandbox; never run customer/AI generated scripts in the main web server.

29.4 Abuse and cost controls

Rate-limit signup, verification sends, chat messages, AI jobs, exports, domain searches, registration attempts and forms. Use bot protection proportionate to observed abuse. Track usage per account and provider spend daily; alert on spikes and sudden changes. A compromised or abusive account can be put into a restricted state without deleting access to invoices, data-export guidance and support.

30. Privacy, Consent and Governance

Collect the minimum data required to create and manage a website. Provide clear purpose, retention, consent and deletion/export mechanisms appropriate to the service's markets and legal obligations.

30.1 Categories of data

Category	Examples	Controls
Identity/account	Email, auth provider ID, name, optional phone	Restrict access; verification and deletion lifecycle.
Business profile	Business name, services, location, contact channels, audience	Owner-controlled; avoid collecting unnecessary sensitive personal information.
Site content	Pages, brand materials, blog drafts, uploaded assets	Tenant isolation, backup and export.
Usage/telemetry	AI job duration/cost, request IDs, error metadata	Avoid raw content in routine logs; documented retention.
Billing	Subscription, invoice, provider reference, payment state	No raw card data; restricted financial access.
Domain registration	Registrant details, domain, term, renewal state	Protect personal contact data; use registrar-supported privacy where eligible.
Consent/preferences	Newsletter opt-in, notification preferences, policy version/time	Auditable opt-in/withdrawal records.
Support/audit	Issue details, privileged action events	Least privilege and a justified retention policy.

30.2 Privacy controls

- Publish a plain-language privacy notice identifying the platform operator, purpose, data classes, recipients/processors, retention and contact path.
- Document processor/vendor relationships and transfer considerations for AI, email, hosting, image and payment vendors.
- Provide account data export and a deletion request workflow with lawful retention exceptions explained.
- Keep newsletter consent separate from service email requirements.
- Minimise prompts and provider logging; do not include full account profiles where only one page field is needed.
- Use sensitive-data minimisation in asset preferences and do not infer personal ethnicity/identity from geolocation.
- Define breach/incident response responsibilities and communication escalation.

30.3 Legal validation

Before launch, obtain appropriate review of applicable Kenyan data protection requirements, consumer billing disclosures, domain reseller obligations, subscription cancellation/refund terms, content licences and email marketing compliance. This TDD is not legal advice. Record the accountable business owner and review date for each policy.

31. Performance and Reliability Targets

Targets are provisional engineering objectives to be tested against actual hosting and traffic patterns. The team should measure the critical paths rather than assume framework defaults are sufficient.

31.1 Initial service objectives (proposed)

Area	Initial objective	Measurement
Dashboard availability	99.5% monthly in MVP; raise target after operational maturity	Synthetic checks and server error rate.
Published site availability	99.9% target once hosting/provider terms support it	Multi-region or external synthetic probes and host platform telemetry.
Home page performance	Aim for Core Web Vitals “good” thresholds on representative mobile samples	LCP <= 2.5s, INP <= 200ms, CLS <= 0.1 at 75th percentile when field data exists; lab testing is diagnostic, not proof.
Preview readiness	Set p50/p95 after measuring first 100+ real builds; target staged progress and no hanging request	Queue delay, stage duration, generation retries and total time.
Webhook processing	Persist verified event quickly; fulfill asynchronously and alert on oldest pending event	Webhook acknowledgement time and queue age.
Export success	At least 99% of supported-feature exports pass smoke test before release	Build validation, broken-link and asset checks.
Data restore	Define recovery point objective and recovery time objective in operations review	Quarterly restore drill and measured recovery time.
Critical security response	Immediate paging/triage for cross-tenant access, domain takeover or billing integrity defects	Incident metrics and postmortem.

31.2 Performance strategy

- Use server-rendered or pre-rendered public content where possible; avoid unnecessary browser JS for static text and SEO.
- Optimise images using Cloudinary transformations; serve correct dimensions and modern formats with responsive `srcset` where supported.
- Lazy-load below-the-fold images and media; avoid loading all five hero images at full resolution at once.
- Limit animation and dashboard bundles; split optional visual effects and avoid layout shift.
- Cache public releases by immutable version and carefully invalidate after publish/domain changes.
- Use database indexes on site ownership, hostname lookup, status, job queue, due renewals and notification unread queries.
- Avoid N+1 queries for page/section/asset graphs; use predictable joins/queries and bounded response payloads.
- Apply job concurrency and connection pooling; load test database and workflow provider under bursts.

31.3 Reliability patterns

Use health checks, timeouts, bounded retries, durable workflow state, circuit breakers, fallback content and rolling deploys where available. User-facing build progress should survive page refresh and device change by reading persisted job state rather than relying only on an in-memory stream. Separate system status from the customer’s site status so a temporary editor outage does not make a successfully published static-like page unnecessarily unavailable.

32. Observability, Audit and Support

Operations must be able to answer: what failed, which site/account was affected, where the workflow stopped, what was charged, and how to recover without asking the customer to repeat the whole build.

32.1 Logs, metrics and traces

Signal	Examples
Structured logs	requestId, jobId, accountId/siteId, stage, provider, outcome, latency, retry, status code
Metrics	AI spend/tokens, generation success/failure, queue age, build time, payment webhook lag, domain readiness, registrar errors, email bounce, export errors
Tracing	Browser request through API, workflow stages, provider calls and publish release
Business metrics	Activation, preview-to-publish conversion, edit frequency, churn, plan usage, support requests
Security signals	Failed login/OTP spikes, tenant access denials, unusual domain changes, webhook signature failures, spend anomalies

32.2 PII and content redaction

Operational logs should prefer IDs and structured error codes over full prompt transcripts, user emails or page body text. If content capture is required for quality analysis, restrict access, document retention and separate it from general application logs. Never log passwords, verification secrets, complete auth tokens, payment card data or provider secret values.

32.3 Audit events

Audit at minimum: login/security configuration changes, account email/phone changes, site ownership changes, publishes and rollbacks, domains added/removed/transferred, registrant updates, billing plan/override/refund, administrator impersonation/support access, asset licence changes, prompt/manifest promotion and secret rotation records (not the secret itself). Audit rows should be append-only for regular application roles.

32.4 Support tools

Support UI should reveal job status, failure code, current provider stage, last retry, safe usage summary, release ID and domain status. Any action that views private customer content or changes a resource must display a reason field and create an audit record. Provide customer-visible diagnostics that avoid exposing infrastructure secrets.

33. Deployment, CI/CD and Environments

Use a repeatable release pipeline with separate development, staging and production environments. Production schema changes, provider keys and domain changes must not be tested against live customer data.

33.1 Environments

Environment	Purpose	Requirements
Local development	Feature work and unit tests	Mock providers by default; local secrets file not committed; seed synthetic data.
CI/test	Lint, type-check, unit/integration/contract tests, manifest/schema validation	Ephemeral credentials, no production secrets, deterministic fixtures.
Staging	End-to-end tests with sandbox payment/registrar or clearly controlled test domains	Separate database/storage/billing sandbox, sample custom domain and email sink.
Production	Real customer websites and billing	Reviewed deployment, monitoring, backups, tested rollback and verified environment settings.

33.2 Release gate

- Lint, type-check and compile pass.
- Database migrations validate and apply in a test environment.
- Unit/integration tests pass, including site/tenant authorization.
- Section manifest and prompt schema validation passes.
- Accessibility and visual regression suite passes for high-use components and key themes.
- Static export compatibility tests and dependency report pass.
- Secret scanning, dependency audit and container/build security checks pass.
- Staging smoke suite verifies signup, generation, preview, publishing, domain mapping, payment simulation and export.
- Production change has owner, deployment notes, monitoring window and rollback plan.

33.3 Rollback strategy

Application deployment rollback should not assume database migration rollback is always safe. Prefer backward-compatible expand/contract migrations and feature flags. Website releases are separately versioned so a code deployment and a customer content rollback are distinct operations. If a new section implementation is defective, disable it for new compositions and maintain compatibility for already published references until migrated.

34. Testing and Release Gates

Test not only code paths but also complete generated sites, exported packages and recovery from third-party failures. AI output is variable, so combine strict deterministic validation with a benchmark of representative prompts and human review.

34.1 Test layers

Layer	Coverage
Unit	Manifest schemas, composition constraints, token budget, slug normalisation, URL safety, domain state transitions, plan entitlements.
Component	Responsive section rendering, long text, empty optional fields, image failure, keyboard interaction, reduced motion.
Integration	Database ownership checks, RLS, queue persistence, provider adapters, upload and transformed image URLs, email templates.
Contract	AI schema outputs, registrar API wrapper, payment webhooks, email events and host platform domain status.
End-to-end	New account -> verify -> brief -> build -> preview -> edit -> checkout -> publish -> custom domain -> export.
Security	Cross-tenant access, XSS/CSRF, prompt injection, webhook spoof/replay, export secret scan, domain takeover scenarios.
Performance	Public pages on mobile, dashboard first load, homepage queries, generation bursts, asset delivery, export build sizes.
AI quality	Fixed industry benchmark, factuality checks, length/count compliance, tone quality, repair rates and regressions by prompt version.
Operational	Queue retry/resume, provider outage, payment succeeds but registrar fails, renewal failure, backup restore, rollback.

34.2 AI evaluation set

Maintain synthetic representative briefs for tourism, restaurants, professional services, trades, retail/product catalogues, personal portfolio and nonprofit/community examples. Include short/long business names, missing facts, unusual punctuation, multilingual terms if supported, false claims requested by the user, prompt-injection text, insufficient images and niche business categories. Track schema validity, unsupported claim rate, page coherence, copy length, section suitability, time and cost.

34.3 Visual regression

Capture screenshots for representative section variants at mobile and desktop viewport sizes using stable fixture data and local/controlled image mocks where feasible. Visual tests should detect layout regressions, not assert that AI-written copy is byte-identical across generations. Separate component snapshots from end-to-end generated site quality evaluations.

34.4 Release acceptance

A release is not accepted solely because it compiles. Every critical path must pass functional, security, responsive, accessibility and failure-recovery checks. Any unresolved critical cross-tenant, domain ownership, payment integrity, data-loss or secret-leak issue blocks production release.

35. Operations, Backup and Disaster Recovery

The platform is a recurring service, so operational practices affect customer trust as directly as design quality. Backups need to be restored successfully in practice, not merely enabled in a dashboard.

35.1 Backup scope

- PostgreSQL data and schema migration records.
- Canonical site configuration, page content and release metadata.
- Prompt and manifest version history stored in source control or durable registry.
- Cloudinary originals where rights and provider terms allow; maintain a recovery strategy for curated media.
- Export build metadata and support/audit data according to retention policy.
- Registrar/domain lifecycle information and payment reconciliation identifiers, excluding provider secrets.

35.2 Recovery objectives

Define a product-level Recovery Point Objective (maximum acceptable data loss) and Recovery Time Objective (maximum acceptable recovery time) with the hosting/database plan owner. Initial suggested discussion targets are an RPO of 24 hours or better and an RTO of 8 hours for the MVP; paid production scale may require tighter targets. These are proposals, not guarantees, and must be validated by the actual database, hosting and backup plan.

35.3 Incident runbook catalogue

Incident	First response	Recovery / follow-up
AI provider degraded	Confirm provider status; throttle new work and expose retryable status	Switch to approved fallback only if tested; resume persisted jobs and reconcile spend.
Database unavailable	Check service health and connection saturation; disable risky migrations	Restore service or fail over; verify no duplicate payment/publish effects.
Domain mapped to wrong site	Immediately disable affected mapping and preserve logs	Verify ownership/mapping history, invalidate caches, investigate takeover root cause.
Payment confirmed but fulfilment failed	Match provider payment and internal order IDs	Retry idempotent fulfilment or issue refund/recovery path; notify customer.
Registrar registration uncertain	Query registrar for domain state before any retry	Avoid duplicate charges/registrations; resolve with registrar support.
Published release broken	Revert site publish pointer to last healthy release	Run regression test, preserve failing artifact and create postmortem.
Cloudinary asset unavailable	Use fallback/asset health checks and identify affected sites	Restore asset or replace binding; inspect provider/account/licence state.
Credential leak	Revoke/rotate credential and assess affected permissions	Audit logs and access, redeploy secrets, notify according to incident policy.
Unexpected AI spend spike	Set concurrency/rate caps and pause anomalous accounts/jobs	Identify prompt loop/abuse, reconcile provider cost and improve guardrails.

35.4 Customer communications

For an incident affecting published sites, communicate what is affected, what remains available, whether customer data is impacted, the current mitigation and the next update channel. Never imply that a domain or website is active when its verification state is unknown. Preserve evidence and conduct a blameless post-incident review for material failures.

36. Delivery Roadmap and Acceptance

Delivery should move from a validated design engine to a reliable self-service pipeline, then add automation of payments and domains. Each phase must produce a usable increment and measurable acceptance evidence.

Phase	Scope	Exit criteria
Phase 0 - Architecture proof	Renderer prototype, section manifest, sample themes, database ownership model, provider proofs of concept	One sample site renders from stored config; same config exports to a working static package; two tenants are isolated.
Phase 1 - Design engine	Core pages, header/footer/CTA, 20-30 curated variants, responsive layout, theme tokens, asset catalogue	Component and mobile visual suite passes; config schema/version strategy documented.
Phase 2 - AI pipeline	Business intake, site plan, per-page content jobs, 3 blog draft jobs, validation/repair and preview	Benchmark reports generated; failed stages resume without rebuilding successful stages; no invented testimonials in tests.
Phase 3 - Accounts/dashboard	Signup, Google/email auth, verification, Home/Build/Websites/Profile, notifications	Access and account recovery suite passes on desktop/mobile; notification links stay tenant-scoped.
Phase 4 - Managed publish	Platform subdomain, release snapshots, publish/rollback, sitemap/metadata	Automated publication and rollback tests; published sites pass health and SEO checks.
Phase 5 - External domains	Registrar adapter, custom hostname verification, TLS readiness, renewal status	Test domain lifecycle using supported test domain; `.co.ke` provider coverage confirmed before advertising.
Phase 6 - Billing/email	Plan catalogue, usage allowance, checkout, verified webhooks, Brevo transaction emails/newsletter consent	No duplicate fulfilment on repeated webhook; receipt and renewal workflow tested end-to-end.
Phase 7 - Export	Static HTML/CSS/JS/assets package, dependency report, README	Package deploys to independent static host; unsupported backend features are flagged.
Phase 8 - Production hardening	Load/security/accessibility tests, backups, on-call runbooks, legal/vendor review	Release gate signed off; no unresolved critical/high-risk issues without written exception.

36.1 Product acceptance definition

- A nontechnical customer can sign up, verify their account, submit a concise brief and see a preview without manipulating source code.
- The generated site has required pages, working navigation, responsive layouts, valid contact paths and appropriate SEO artefacts.
- Every selected component/version is registered and validated.
- AI content remains within schema and does not fabricate customer proof points.
- Published config is versioned; a failed publish does not replace the previously healthy release.
- A customer can export a portable package and deploy it elsewhere, with backend-dependent features disclosed.
- Billing/registration states are based on provider-verified results and idempotent fulfilment.
- The product behaves coherently on mobile and desktop and respects accessibility/reduced-motion requirements.
- Privacy notice, terms, support contact, data export/deletion process and policy for domains after cancellation are published.

37. Functional Requirement Specifications

Each specification has a unique ID and priority. MUST means required for the defined MVP; SHOULD means valuable but can be gated by launch scope; LATER indicates a planned extension that should not block initial release. Every MUST requirement must map to one or more tickets and tests before release.

FR-001 - Account registration

Priority: MUST Requirement family: Functional

Purpose

Allow a visitor to create a platform account using an email address and password.

Primary workflow

Validate email syntax, password policy and anti-abuse controls; create a pending/unverified account and issue verification message.

Rules and invariants

Do not enable publishing, domain purchase or paid fulfilment until email verification succeeds. Avoid revealing account existence through error wording.

Failure and recovery behaviour

Mail-provider failure leaves a recoverable pending account; resend is rate limited.

Acceptance criteria

A valid new user receives a verification route and is directed to a helpful verify-email screen; duplicate registrations do not create duplicate accounts.

Dependencies and implementation notes

Auth provider, Brevo transactional email, rate limiting.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-001 in the delivery tracker.

FR-002 - Google sign-in

Priority: MUST Requirement family: Functional

Purpose

Offer a Google OAuth option when configured.

Primary workflow

Use server-validated OAuth callback, verify provider identity and safe email/account-linking policy, then create a platform session.

Rules and invariants

Never trust a client-supplied Google email or ID; account linking to an existing account requires a safe verification path.

Failure and recovery behaviour

OAuth denial, expired code or misconfigured callback returns a safe recoverable error.

Acceptance criteria

A verified Google identity can sign in and access only its own account resources; invalid state/nonce is rejected.

Dependencies and implementation notes

Auth library, OAuth console configuration, callback URL allowlist.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-002 in the delivery tracker.

FR-003 - Email/password sign-in

Priority: MUST Requirement family: Functional

Purpose

Support verified accounts signing in with email/password.

Primary workflow

Delegate password validation and session creation to vetted auth implementation; throttle failures and create security telemetry.

Rules and invariants

Do not store plaintext passwords; apply appropriate hashing through the auth provider; generic failure messages reduce enumeration.

Failure and recovery behaviour

Repeated failures may trigger delay or challenge; no response reveals password or whether another tenant exists.

Acceptance criteria

Correct credentials create a secure session; wrong credentials do not disclose sensitive account status.

Dependencies and implementation notes

Authentication library, session cookie configuration.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-003 in the delivery tracker.

FR-004 - Email verification

Priority: MUST Requirement family: Functional

Purpose

Verify email ownership before sensitive publishing/billing actions.

Primary workflow

Send time-limited, single-use verification link or code; validate at server; mark verified and invalidate used token.

Rules and invariants

Rate-limit issuance and redemption; new verification events may invalidate older tokens; verification code is never logged.

Failure and recovery behaviour

Expired/used token offers an explicit resend flow; provider outage is shown as retryable.

Acceptance criteria

A valid token verifies exactly one account once; replay and expired token are rejected.

Dependencies and implementation notes

Auth service, transactional email, token store.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-004 in the delivery tracker.

FR-005 - Password reset

Priority: MUST Requirement family: Functional

Purpose

Enable secure account recovery.

Primary workflow

User requests reset; system sends neutral response and a time-limited single-use reset token; upon valid reset revoke relevant sessions.

Rules and invariants

Never email current passwords; do not expose account existence; enforce attempt limits and password policy.

Failure and recovery behaviour

Expired or already-used token cannot change password; rate limits are visible without disclosing account data.

Acceptance criteria

A successfully reset password allows login and invalidates the reset token; security event is recorded.

Dependencies and implementation notes

Auth provider, email delivery, audit events.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-005 in the delivery tracker.

FR-006 - Session management

Priority: MUST Requirement family: Functional

Purpose

Keep authenticated sessions secure and revocable.

Primary workflow

Issue secure cookie, rotate after auth/privilege changes, enforce idle/absolute expiry and support logout/revocation.

Rules and invariants

Use Secure/HttpOnly/SameSite settings as appropriate; do not place tokens in URLs or localStorage by default.

Failure and recovery behaviour

Expired session routes user to sign-in and preserves safe return path; logout works across protected pages.

Acceptance criteria

Session fixation/reuse tests fail safely; logout and credential reset revoke sessions according to policy.

Dependencies and implementation notes

Auth configuration, server middleware, security tests.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-006 in the delivery tracker.

FR-007 - Personal profile and preferences

Priority: MUST Requirement family: Functional

Purpose

Let users manage permitted personal information and preferences.

Primary workflow

Expose validated fields for display name, optional phone, timezone, language, communication preferences and account security.

Rules and invariants

Only update allowlisted fields; email/phone changes require verification; sensitive operations may require re-auth.

Failure and recovery behaviour

Validation errors remain attached to fields; failed save leaves prior data intact.

Acceptance criteria

User changes persist and are visible on next sign-in; another user cannot read or update the profile.

Dependencies and implementation notes

User table, profile UI, verification.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-007 in the delivery tracker.

FR-008 - Optional phone verification

Priority: SHOULD Requirement family: Functional

Purpose

Support phone verification when product policy requires added assurance.

Primary workflow

Issue SMS OTP through a supported messaging provider; verify within limited time and attempts.

Rules and invariants

Must be optional for ordinary onboarding at launch unless risk/legal analysis says otherwise; protect phone data.

Failure and recovery behaviour

Provider unavailable or OTP expiry allows retry with limits; do not create verified status merely on send.

Acceptance criteria

Valid OTP verifies the intended number once; expired/replayed attempts are rejected.

Dependencies and implementation notes

SMS provider decision, user consent, rate limiting.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-008 in the delivery tracker.

FR-009 - Resource authorisation

Priority: MUST Requirement family: Functional

Purpose

Enforce ownership and role permissions on every protected API.

Primary workflow

Resolve authenticated account and verify membership/ownership for each resource before reads and mutations.

Rules and invariants

Client-provided IDs are not authorization; admin/support permissions must be explicit and auditable.

Failure and recovery behaviour

Denied cross-tenant requests return safe errors and create relevant security telemetry.

Acceptance criteria

Automated tests cover cross-tenant GET/POST/PATCH/DELETE for each resource family.

Dependencies and implementation notes

Identity model, site membership, RLS/application policy.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-009 in the delivery tracker.

FR-010 - Account data export and deletion

Priority: MUST Requirement family: Functional

Purpose

Allow account holders to request data export and deletion.

Primary workflow

Verify requester, assemble supported personal/account content into a secure export, or queue deletion workflow with legal-retention exceptions.

Rules and invariants

Explain effects on active domains, billing records and published websites before irreversible deletion; do not delete mandatory transaction records prematurely.

Failure and recovery behaviour

Export job failures can resume; deletion may be blocked pending a user-confirmed domain/ownership decision.

Acceptance criteria

User can download own data only; deletion request is audited and processed under documented retention policy.

Dependencies and implementation notes

Privacy policy, data inventory, background jobs.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-010 in the delivery tracker.

FR-011 - Home dashboard

Priority: MUST Requirement family: Functional

Purpose

Provide a useful landing screen after sign-in.

Primary workflow

Show primary Build action, owned websites and their states, recent activity, notifications and billing/domain alerts where relevant.

Rules and invariants

Never show another tenant’s sites; hide irrelevant setup cards after completion; ensure status comes from persisted data.

Failure and recovery behaviour

API loading/failure provides retry and does not falsely label a site live.

Acceptance criteria

First-time user understands the next step; returning user can open a site or continue a job.

Dependencies and implementation notes

Site APIs, workflow status, notifications.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-011 in the delivery tracker.

FR-012 - Responsive application navigation

Priority: MUST Requirement family: Functional

Purpose

Use desktop sidebar and mobile bottom navigation.

Primary workflow

Desktop/laptop shows sidebar destinations; mobile shows Home, Build, Websites and Profile bottom navigation; notifications are in the top bar.

Rules and invariants

Navigation is keyboard/touch accessible, adapts at tested breakpoints and does not obscure safe areas or content.

Failure and recovery behaviour

When a destination is unavailable by plan, explain why rather than silently redirecting.

Acceptance criteria

All primary destinations are reachable at mobile and desktop widths with visible active state and accessible labels.

Dependencies and implementation notes

Design system, responsive tests.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-012 in the delivery tracker.

FR-013 - Notification centre

Priority: MUST Requirement family: Functional

Purpose

Display important account, build, billing and domain events.

Primary workflow

Persist notification records, show unread count, allow open/read/dismiss actions subject to product rules.

Rules and invariants

Each notification must be scoped to its recipient and link to a resource they can access; create event idempotently.

Failure and recovery behaviour

Missing/deleted target shows safe detail or fallback, not another tenant’s content.

Acceptance criteria

Mark-read updates survive refresh; unread total matches persisted records.

Dependencies and implementation notes

Notification model, job/event system, route guards.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-013 in the delivery tracker.

FR-014 - Website creation and ownership

Priority: MUST Requirement family: Functional

Purpose

Create, list, update, archive and manage owned websites.

Primary workflow

Create site record with unique platform slug, owner/membership, lifecycle state and initial config; list only accessible sites.

Rules and invariants

Unique slug/domain constraints are server enforced; deleting/archive requires policy check for domain and billing entitlements.

Failure and recovery behaviour

Slug conflict returns alternatives; partial creation is rolled back or resumable.

Acceptance criteria

New site is tied to correct account and cannot be accessed by other accounts.

Dependencies and implementation notes

Site schema, authz, plan quotas.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-014 in the delivery tracker.

FR-015 - Business intake

Priority: MUST Requirement family: Functional

Purpose

Collect minimum facts needed to build a credible first version.

Primary workflow

Capture industry, business name, location/market where relevant, offerings, intended audience, tone, key contacts, brand assets/preferences and conversion goal.

Rules and invariants

Reuse supplied data and provide sensible defaults; distinguish optional from essential fields; do not request irrelevant sensitive data.

Failure and recovery behaviour

Draft intake autosaves; invalid contacts are highlighted and not silently used.

Acceptance criteria

A brief can be saved, edited and submitted; missing essential values trigger only targeted follow-up.

Dependencies and implementation notes

BusinessProfile schema, chat UI, validation.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-015 in the delivery tracker.

FR-016 - Necessary follow-up questions

Priority: MUST Requirement family: Functional

Purpose

Ask only questions that materially affect output quality or correctness.

Primary workflow

Planner detects missing facts with high impact and asks concise multiple choice or short answer prompts; store answers in brief.

Rules and invariants

Do not repeat questions answered in intake; low-impact fields use safe defaults; user can skip optional choices.

Failure and recovery behaviour

If user cannot answer, fallback to neutral content or mark missing; build should not dead-end.

Acceptance criteria

Test briefs produce limited relevant questions; repeated answers are not requested.

Dependencies and implementation notes

Prompt planner, brief completeness policy.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-016 in the delivery tracker.

FR-017 - Site strategy plan

Priority: MUST Requirement family: Functional

Purpose

Generate a valid high-level site plan before page copy.

Primary workflow

Create structured plan with purpose, route list, visual direction, section IDs, content jobs, CTA goals and site-specific constraints.

Rules and invariants

Plan can select only registered pages/sections/themes; server validates IDs/counts/allowed page types before continuing.

Failure and recovery behaviour

Invalid plan is repaired within bounded attempts or surfaced with actionable error.

Acceptance criteria

A plan is persisted/versioned, includes required core pages and passes registry validation before content jobs launch.

Dependencies and implementation notes

AI adapter, registry, schema validator.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-017 in the delivery tracker.

FR-018 - Route and sitemap planning

Priority: MUST Requirement family: Functional

Purpose

Create sensible routes and navigation from approved pages.

Primary workflow

Generate canonical slugs for core pages and optional pages; build navigation labels and internal route references.

Rules and invariants

Slug normalisation/uniqueness is deterministic; reserved system paths are prohibited; subsequent slug changes need redirects.

Failure and recovery behaviour

Conflicting slugs are rejected or resolved before publishing; no broken reference is silently emitted.

Acceptance criteria

All planned routes are unique, accessible by nav/internal links where intended and supported by renderer/export.

Dependencies and implementation notes

SitePlan schema, route resolver, SEO module.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-018 in the delivery tracker.

FR-019 - Theme choice and token resolution

Priority: MUST Requirement family: Functional

Purpose

Select an appropriate supported theme and store theme tokens.

Primary workflow

Use customer brand inputs or supported stylistic preferences; resolve palette, typography, shapes, spacing and motion preset.

Rules and invariants

AI can only propose known theme IDs and constrained tokens; contrast/compatibility validation is mandatory.

Failure and recovery behaviour

Incompatible colour/font selection falls back to nearest valid theme and records why.

Acceptance criteria

Theme tokens render on all required pages and pass contrast and responsive tests.

Dependencies and implementation notes

Theme registry, design token validators.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-019 in the delivery tracker.

FR-020 - Section composition

Priority: MUST Requirement family: Functional

Purpose

Choose compatible sections and arrange them into coherent pages.

Primary workflow

Filter by page type, industry, goal, media capacity, theme support and composition rules; order sections and persist seed.

Rules and invariants

Must preserve required hero/CTA/contact paths; no cross-site comparison; within-site visual repetition rules apply.

Failure and recovery behaviour

If no suitable candidates exist, use a tested fallback component or stop before content generation.

Acceptance criteria

The same plan/version/seed is reproducible; every section ID resolves and page composition passes constraints.

Dependencies and implementation notes

Section registry, site plan and seeded selector.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-020 in the delivery tracker.

FR-021 - Seeded diversity without cross-site scans

Priority: MUST Requirement family: Functional

Purpose

Make sites varied without querying all other customer compositions.

Primary workflow

Use weighted selection, site-specific seed, theme combinations and within-page/page-to-page structural diversity.

Rules and invariants

No routine global comparison of site compositions; no promise that no two sites can ever resemble one another.

Failure and recovery behaviour

Insufficient eligible variants use compatible defaults and record limited diversity.

Acceptance criteria

Load tests show composition cost scales with candidate registry size, not tenant count; repeated run remains stable.

Dependencies and implementation notes

Composition algorithm, registry tags.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-021 in the delivery tracker.

FR-022 - Section registry management

Priority: MUST Requirement family: Functional

Purpose

Register, validate, release and deprecate section components.

Primary workflow

Developer registers ID/version, schema, page/industry compatibility, media slots, responsive rules, animation/export support and fixtures.

Rules and invariants

Only released manifests may be selected by production plans; released manifests are immutable or versioned.

Failure and recovery behaviour

Invalid manifest blocks CI release; deprecation cannot break existing live release without migration.

Acceptance criteria

CI verifies manifest uniqueness/schema/component existence and all released site references remain resolvable.

Dependencies and implementation notes

Versioned code registry, CI pipeline.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-022 in the delivery tracker.

FR-023 - Section content schemas

Priority: MUST Requirement family: Functional

Purpose

Define required fields, item counts, lengths and field types per section.

Primary workflow

Each field schema controls data type, word/character guidance, required state, maximum items, URL validation and sanitisation.

Rules and invariants

Runtime validators are authoritative; prompt length instructions are not enough; never silently truncate legal/important copy.

Failure and recovery behaviour

Invalid fields trigger targeted repair or explicit editor warning.

Acceptance criteria

Test fixtures cover missing, empty, overlong and hostile content for every released section schema.

Dependencies and implementation notes

Zod/JSON schemas, manifest registry, prompt builder.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-023 in the delivery tracker.

FR-024 - Section version compatibility

Priority: MUST Requirement family: Functional

Purpose

Keep existing site configurations valid as components evolve.

Primary workflow

Store component ID/version in each instance; maintain compatible renderer or a migration for breaking changes.

Rules and invariants

Do not delete code referenced by published releases until usage is migrated or retention window passes.

Failure and recovery behaviour

A missing legacy version blocks deployment or invokes explicit migration; it cannot fall back silently to a different layout.

Acceptance criteria

Old release can be rendered and rolled back after a new component release.

Dependencies and implementation notes

Manifest registry, release tests, migration tooling.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-024 in the delivery tracker.

FR-025 - Per-page content generation

Priority: MUST Requirement family: Functional

Purpose

Generate page content using page-specific prompt context and section schemas.

Primary workflow

Create independent jobs for homepage, About, Services/Products, Contact, Blog index and optional pages; share approved business facts.

Rules and invariants

Every generation must reference prompt/version/schema inputs; no model HTML or arbitrary component code is accepted.

Failure and recovery behaviour

Transient errors retry by stage; invalid output goes through bounded repair or clear failure state.

Acceptance criteria

Each page payload validates independently and can be regenerated without overwriting unrelated pages.

Dependencies and implementation notes

AI orchestration, schemas, brief/site plan.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-025 in the delivery tracker.

FR-026 - Staged workflow execution

Priority: MUST Requirement family: Functional

Purpose

Coordinate multi-stage generation as durable jobs.

Primary workflow

Persist site plan, spawn dependency-aware page/article jobs, validate, bind assets, build preview and await review.

Rules and invariants

Completed stages are reused on downstream retry; side effects are idempotent; user sees persisted progress.

Failure and recovery behaviour

Worker restart resumes from last completed step; permanent failure identifies stage and preserves successful outputs.

Acceptance criteria

Integration test simulates worker crash after each stage without duplicate charge or lost output.

Dependencies and implementation notes

Workflow engine, database job ledger.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-026 in the delivery tracker.

FR-027 - Three initial blog drafts

Priority: SHOULD Requirement family: Functional

Purpose

Create three starter article drafts appropriate to the industry and initial site.

Primary workflow

Plan suitable topics, enqueue one article job per topic, create individual article records with metadata and internal links.

Rules and invariants

Drafts are not published automatically by default; topics must fit supplied business facts and not make regulated unsupported claims.

Failure and recovery behaviour

Insufficient brief asks for input or produces conservative drafts; one failed post does not fail all other posts.

Acceptance criteria

Three separate draft records appear when eligible; each can be edited, deleted or published independently.

Dependencies and implementation notes

Blog schema, article prompts, allowance metering.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-027 in the delivery tracker.

FR-028 - Factuality and testimonial guardrails

Priority: MUST Requirement family: Functional

Purpose

Avoid fabricated proof points and unsupported claims.

Primary workflow

Prompt uses fact ledger and source material; validation identifies high-risk fields and missing evidence.

Rules and invariants

No synthetic testimonials, awards, rating values, client logos, certifications, staff names, physical addresses or guarantees.

Failure and recovery behaviour

Unsupported content is removed, flagged for owner input or replaced by a different section.

Acceptance criteria

Adversarial tests asking to invent reviews/awards do not produce fabricated facts in publishable output.

Dependencies and implementation notes

Prompt contract, validation rubric, content QA.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-028 in the delivery tracker.

FR-029 - Structured-output validation and repair

Priority: MUST Requirement family: Functional

Purpose

Validate every model result against an explicit schema.

Primary workflow

Parse JSON, validate schema, lengths/counts/IDs/URLs and run permitted semantic checks; request a targeted repair on fixable errors.

Rules and invariants

Retries are bounded; model output never bypasses server validation; schema failures cannot publish.

Failure and recovery behaviour

After retry exhaustion, job is failed with a safe reason and successful earlier stages remain saved.

Acceptance criteria

Malformed JSON, extra fields, unsafe URL schemes and invalid IDs are rejected in automated tests.

Dependencies and implementation notes

Provider structured output, Zod, workflow retries.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-029 in the delivery tracker.

FR-030 - Chat intent routing

Priority: MUST Requirement family: Functional

Purpose

Map natural-language requests to a supported action category.

Primary workflow

Classify content edit, style change, image swap, insert/remove/reorder, page/blog creation, publish/domain/billing action or unsupported request.

Rules and invariants

Action handler is fixed and allowlisted; model cannot grant permissions, issue payment or register domain directly.

Failure and recovery behaviour

Low-confidence intent asks a clarification or presents supported choices rather than taking an irreversible action.

Acceptance criteria

Tests verify prompt injection cannot cause privileged side effects; each allowed intent has a typed request.

Dependencies and implementation notes

Conversation service, tool allowlist, authz.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-030 in the delivery tracker.

FR-031 - Targeted section editing

Priority: MUST Requirement family: Functional

Purpose

Edit only affected fields/sections when possible.

Primary workflow

Resolve target page and section, create change set against expected revision, regenerate selected content or update supported tokens/asset ID.

Rules and invariants

Preserve unrelated approved content and stable composition; optimistic concurrency prevents overwriting newer changes.

Failure and recovery behaviour

Stale target asks user to refresh/rebase; partial update is not applied on version conflict.

Acceptance criteria

Editing a headline leaves other page content and asset bindings unchanged unless explicitly requested.

Dependencies and implementation notes

Site revisions, intent router, schema validator.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-031 in the delivery tracker.

FR-032 - Confirmation for destructive/high-impact actions

Priority: MUST Requirement family: Functional

Purpose

Require explicit confirmation for destructive or costly actions.

Primary workflow

Show clear summary before deleting page/section, changing slug/domain, purchasing/renewing domain, cancelling plan or publishing high-impact changes.

Rules and invariants

AI message alone is not a confirmation; require authenticated action tied to target resource and quoted cost/effect.

Failure and recovery behaviour

Expired quote or changed target requires reconfirmation.

Acceptance criteria

No protected action occurs until a valid confirmation token/action is recorded.

Dependencies and implementation notes

Chat router, billing/domain services, audit.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-032 in the delivery tracker.

FR-033 - Revision history

Priority: MUST Requirement family: Functional

Purpose

Keep drafts and published revisions recoverable.

Primary workflow

Create revision records for material config/content changes; show author/time/summary and diff where possible.

Rules and invariants

Published releases are immutable; rollback creates a new release or pointer event and is audited.

Failure and recovery behaviour

Revision conflict is handled explicitly; missing asset/version blocks rollback with clear explanation.

Acceptance criteria

User can restore a prior revision without losing audit history; live and draft versions are distinguishable.

Dependencies and implementation notes

Versioned site config, releases and asset bindings.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-033 in the delivery tracker.

FR-034 - Preview generation

Priority: MUST Requirement family: Functional

Purpose

Show a private preview before publishing.

Primary workflow

Build preview from a specific draft revision, run checks, generate unguessable or authenticated URL and show status.

Rules and invariants

Preview is noindex, never changes public release pointer and respects private access controls.

Failure and recovery behaviour

Build failure preserves draft and shows retryable/permanent status; preview expiry is handled.

Acceptance criteria

Preview reflects the identified revision and remains isolated from anonymous visitors.

Dependencies and implementation notes

Renderer, build worker, access tokens.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-034 in the delivery tracker.

FR-035 - Publish approval and release

Priority: MUST Requirement family: Functional

Purpose

Publish an approved valid site revision.

Primary workflow

Check ownership/plan/required domain state; execute checks; write immutable release; atomically switch pointer; smoke test and notify owner.

Rules and invariants

No publish on failing critical validation; no client-side success-only payment check; old release remains active on failure.

Failure and recovery behaviour

If final check fails, preserve previous site and show issue; release job can safely resume.

Acceptance criteria

Publish operation yields one unique release and health check; failure does not blank previously live site.

Dependencies and implementation notes

Release model, job orchestration, hosting/domain APIs.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-035 in the delivery tracker.

FR-036 - Rollback published site

Priority: MUST Requirement family: Functional

Purpose

Recover from a bad release.

Primary workflow

Allow authorised owner/admin to select a known-good release and create a new activation event after compatibility checks.

Rules and invariants

Rollback is audited and does not erase failed release; asset/domain dependencies must remain valid.

Failure and recovery behaviour

Unavailable old component/assets require migration or fail-safe warning before rollback.

Acceptance criteria

Prior functional site can be restored; rollback result is verified at public route.

Dependencies and implementation notes

Release history, component version retention, caches.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-036 in the delivery tracker.

FR-037 - Page creation and removal

Priority: MUST Requirement family: Functional

Purpose

Add or remove supported pages with safe route handling.

Primary workflow

Select an approved page type/section skeleton, generate content, add navigation where requested, and update sitemap after publish.

Rules and invariants

Removing published route should create redirect or deliberate 404 policy; core pages cannot accidentally disappear below plan requirements.

Failure and recovery behaviour

Slug conflict/plan limit blocks creation with options; failed generation does not create half-page in navigation.

Acceptance criteria

New page validates and links correctly; removed page no longer in sitemap and configured redirect works.

Dependencies and implementation notes

Route planner, plan entitlements, SEO module.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-037 in the delivery tracker.

FR-038 - Sticky header and navigation

Priority: MUST Requirement family: Functional

Purpose

Provide clear navigation on generated sites.

Primary workflow

Render tested header variants with primary nav, logo/brand link, CTA where useful, desktop and mobile states.

Rules and invariants

Sticky behaviour must not obscure anchors/content; keyboard focus and menu state are accessible.

Failure and recovery behaviour

Missing logo uses text fallback; long labels collapse into mobile menu safely.

Acceptance criteria

Navigation works by mouse, touch and keyboard; active route and focus are clear on supported widths.

Dependencies and implementation notes

Section/component registry, theme tokens, a11y tests.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-038 in the delivery tracker.

FR-039 - Four-column adaptive footer

Priority: MUST Requirement family: Functional

Purpose

Provide detailed footer with four desktop columns.

Primary workflow

Compose brand summary, primary links, offerings/resources and contact/social/newsletter content when configured.

Rules and invariants

Collapse to fewer columns on mobile; do not invent contact details; external links must be valid and safe.

Failure and recovery behaviour

Missing column data uses a compatible simplified layout rather than blank placeholders.

Acceptance criteria

Footer works at mobile/tablet/desktop sizes, links resolve, and contact details match the business brief.

Dependencies and implementation notes

Global shell, business profile, responsive component library.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-039 in the delivery tracker.

FR-040 - Floating WhatsApp button

Priority: SHOULD Requirement family: Functional

Purpose

Add a floating WhatsApp contact action where configured.

Primary workflow

Validate destination number/link, encode optional prefilled message, and render accessible labelled button at responsive position.

Rules and invariants

Never show placeholder destination; do not overlap essential controls or ignore mobile safe areas.

Failure and recovery behaviour

Invalid/absent number hides the button and prompts owner to configure it.

Acceptance criteria

Valid button opens expected WhatsApp destination; screen reader label is meaningful; mobile UI is not obstructed.

Dependencies and implementation notes

Business profile, URL validator, responsive tests.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-040 in the delivery tracker.

FR-041 - Contact form

Priority: MUST Requirement family: Functional

Purpose

Capture customer enquiries on the published website.

Primary workflow

Render schema-defined fields, validate client/server, protect against spam, persist enquiry or deliver via a configured supported destination.

Rules and invariants

Do not claim delivery unless backend/provider confirms; disclose data purpose; rate-limit and sanitize inputs.

Failure and recovery behaviour

Provider/database outage produces an understandable error and safe retry path; avoid duplicate submission via idempotency token.

Acceptance criteria

Valid enquiry is recorded/delivered once and appropriate owner notification is triggered.

Dependencies and implementation notes

Form service, storage/email provider, privacy policy.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-041 in the delivery tracker.

FR-042 - Services/products page

Priority: MUST Requirement family: Functional

Purpose

Present the business's approved service/product entries.

Primary workflow

Render configurable item count, cards or detailed service blocks, supporting copy and appropriate enquiry CTAs.

Rules and invariants

No invented prices, stock, guarantees or purchase capabilities; ecommerce is out of scope unless explicitly added.

Failure and recovery behaviour

Missing entries trigger a reasonable onboarding question or explanatory empty state.

Acceptance criteria

All items come from confirmed brief/owner edits and each configured CTA resolves.

Dependencies and implementation notes

Business brief, content schemas, route/navigation.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-042 in the delivery tracker.

FR-043 - Testimonials and social proof policy

Priority: MUST Requirement family: Functional

Purpose

Use customer-provided evidence without generating fictional proof.

Primary workflow

Show only owner-entered/approved quote and attribution records; require consent/usage permission representation when needed.

Rules and invariants

If none exists, omit testimonial component or choose alternative evidence-free section; do not create star ratings.

Failure and recovery behaviour

Invalid/missing attribution shows anonymous approved quote only if customer explicitly selects that mode.

Acceptance criteria

Generated site with no testimonial input contains no invented quotes, names, logos, awards or rating schema.

Dependencies and implementation notes

Content validator, section composition, review policy.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-043 in the delivery tracker.

FR-044 - Blog editing and publication

Priority: MUST Requirement family: Functional

Purpose

Support article drafts, editing, publishing, unpublishing and slug changes.

Primary workflow

Edit fields, preview article, manage status and featured asset; publish only through validation.

Rules and invariants

Drafts excluded from sitemap/public access; slug changes require redirect/canonical update; AI generated byline must not imply a real person without approval.

Failure and recovery behaviour

Validation failure keeps prior live article; duplicate slug prompts replacement.

Acceptance criteria

Publishing adds page to blog index/sitemap and unpublishing removes it from index per defined status policy.

Dependencies and implementation notes

Article model, releases, SEO and route service.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-044 in the delivery tracker.

FR-045 - SEO metadata generation

Priority: MUST Requirement family: Functional

Purpose

Provide metadata appropriate to each public route.

Primary workflow

Generate title/description/canonical/social metadata from page facts and check length/uniqueness guidance.

Rules and invariants

No keyword-stuffing guarantees; metadata cannot include unverified location or ratings; preview metadata is nonindexable.

Failure and recovery behaviour

If AI output fails checks, use safe deterministic fallback and allow owner edit.

Acceptance criteria

Every public page has valid title, description policy and canonical URL after custom domain activation.

Dependencies and implementation notes

Page renderer, SEO schemas, domain canonical resolver.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-045 in the delivery tracker.

FR-046 - Sitemap and robots

Priority: MUST Requirement family: Functional

Purpose

Expose appropriate sitemap.xml and robots.txt for hosted sites and exports.

Primary workflow

Build sitemap from public canonical routes and public published articles; exclude previews, drafts and private routes.

Rules and invariants

Respect intentional noindex/private state; custom domain changes update canonical host; do not list redirects/404 routes as canonical pages.

Failure and recovery behaviour

Build failure blocks publish or serves previous valid sitemap with alert depending on release policy.

Acceptance criteria

Sitemap contains expected routes only; robots file does not expose internal dashboard and preview paths.

Dependencies and implementation notes

Route inventory, publish release, domain state.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-046 in the delivery tracker.

FR-047 - Structured data

Priority: SHOULD Requirement family: Functional

Purpose

Emit appropriate structured data when factual eligibility is met.

Primary workflow

Select schema type for business/article/site context and fill only verified fields.

Rules and invariants

No fabricated aggregateRating, review, address, price or credential data; must remain valid JSON-LD.

Failure and recovery behaviour

Invalid schema is omitted or repaired; errors visible in build check report.

Acceptance criteria

Automated validator confirms schema syntax and all values map to approved facts.

Dependencies and implementation notes

Business brief, page schema, SEO validation.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-047 in the delivery tracker.

FR-048 - Asset ingestion and approval

Priority: MUST Requirement family: Functional

Purpose

Import images into a curated catalogue safely.

Primary workflow

Upload to controlled Cloudinary account, inspect format/size/dimensions, capture source/licence and route to review.

Rules and invariants

Only approved assets can be selected by production generator; provider secrets remain server-only.

Failure and recovery behaviour

Upload/API failure does not create active catalogue entry; duplicate ingestion is idempotent by provider ID/hash.

Acceptance criteria

Approved asset has valid metadata, provenance and retrievable transformed URL.

Dependencies and implementation notes

Cloudinary, Asset DB, curator UI and validation worker.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-048 in the delivery tracker.

FR-049 - Asset metadata and tags

Priority: MUST Requirement family: Functional

Purpose

Store searchable descriptors for each image.

Primary workflow

Record subject, industry, section use, orientation, aspect ratio, style, context, focal point, alt text and licence fields.

Rules and invariants

Metadata must distinguish fact from inference; licence/source is not optional for production assets unless internally owned and recorded.

Failure and recovery behaviour

Missing critical licence fields prevent approval; low-confidence tags need review.

Acceptance criteria

Asset search can return assets filtered by allowed section and aspect ratio; catalogue audit detects incomplete records.

Dependencies and implementation notes

Asset model, search indexes, curator review.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-049 in the delivery tracker.

FR-050 - Asset selection and binding

Priority: MUST Requirement family: Functional

Purpose

Bind curated images to compatible section media slots.

Primary workflow

Filter approved/licensed assets, rank based on subject/context/style/aspect/representation preference and persist asset IDs.

Rules and invariants

No arbitrary URLs from AI; avoid repeating same image within a site unless intentionally reused.

Failure and recovery behaviour

If no valid candidate exists, use neutral fallback or omit optional image and report missing media.

Acceptance criteria

Every bound asset is approved, accessible and matches component media slot constraints.

Dependencies and implementation notes

Asset catalogue, component manifests, composition seed.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-050 in the delivery tracker.

FR-051 - Image failure fallback

Priority: MUST Requirement family: Functional

Purpose

Avoid visibly broken images when media is unavailable.

Primary workflow

Use onError fallback for component asset slots, resilient URLs, reserved aspect ratios and optional retry/health check.

Rules and invariants

Fallback must not show a misleading subject or make a false factual claim; alt text remains appropriate.

Failure and recovery behaviour

Unavailable hero asset switches to another approved asset or safe background; layout does not collapse.

Acceptance criteria

Simulated HTTP error produces stable layout and no broken-image icon across tested components.

Dependencies and implementation notes

Renderer, asset resolver, error monitoring.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-051 in the delivery tracker.

FR-052 - Image licence tracking

Priority: MUST Requirement family: Functional

Purpose

Respect media licence and attribution requirements.

Primary workflow

Store licence, source, expiry, allowed use, attribution and export rights; review expiring/revoked assets.

Rules and invariants

Expired/revoked asset cannot be newly selected; exports only include assets permitted for export/redistribution.

Failure and recovery behaviour

Licence status unknown causes hold/replace workflow rather than silent continuing selection.

Acceptance criteria

Asset export checks permission and produces attribution inventory when required.

Dependencies and implementation notes

Asset catalogue, curator process, export validator.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-052 in the delivery tracker.

FR-053 - Regional and representation context

Priority: SHOULD Requirement family: Functional

Purpose

Select imagery that fits business context and owner preferences.

Primary workflow

Use explicit market, environment, subject and representation preferences to rank images; permit owner override.

Rules and invariants

Do not hard-code Africa=Black, Asia=Asian, Europe/US=White; do not infer an individual’s ethnicity from location.

Failure and recovery behaviour

If preference data is absent, use contextually appropriate inclusive defaults and ask only when relevance matters.

Acceptance criteria

Test across markets shows locally relevant contexts without forced demographic mapping or unsupported identity claims.

Dependencies and implementation notes

Asset tags, onboarding options, representation guidelines.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-053 in the delivery tracker.

FR-054 - Self-contained image export

Priority: MUST Requirement family: Functional

Purpose

Package permitted image assets for export when requested.

Primary workflow

Resolve each referenced asset, verify export rights and download approved rendition into package or retain disclosed remote dependency.

Rules and invariants

Never transfer assets where licence disallows redistribution; asset manifest lists excluded/remote media and reasons.

Failure and recovery behaviour

Unavailable permitted asset blocks or warns according to criticality; customer receives actionable report.

Acceptance criteria

Static export passes local asset existence test and includes attribution information where required.

Dependencies and implementation notes

Cloudinary, licence data, export job.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-054 in the delivery tracker.

FR-055 - GSAP animation presets

Priority: SHOULD Requirement family: Functional

Purpose

Apply tested animation patterns to supported sections.

Primary workflow

Manifest selects named preset such as fade-up, stagger cards, reveal or hero crossfade; preset implementation owns timeline lifecycle.

Rules and invariants

No AI-generated arbitrary JavaScript; component renders readable content if animation never runs.

Failure and recovery behaviour

Hydration mismatch or animation exception falls back to static visible content and is reported.

Acceptance criteria

Preset has tests for route change, unmount cleanup and reduced-motion state.

Dependencies and implementation notes

GSAP, registry allowlist, frontend tests.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-055 in the delivery tracker.

FR-056 - Reduced motion

Priority: MUST Requirement family: Functional

Purpose

Respect user preference for reduced motion.

Primary workflow

Read prefers-reduced-motion and provide static/low-motion variant; allow user/site setting where appropriate.

Rules and invariants

Animation is never the sole carrier of information or necessary navigation feedback.

Failure and recovery behaviour

If preference API unavailable, use conservative default and keep core content visible.

Acceptance criteria

Automated test with reduced-motion emulation confirms no required content stays hidden or animated excessively.

Dependencies and implementation notes

Design tokens, GSAP/CSS presets, accessibility QA.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-056 in the delivery tracker.

FR-057 - Mobile-first generated sites

Priority: MUST Requirement family: Functional

Purpose

Ensure all public site templates work from small viewports upward.

Primary workflow

Use mobile-first CSS and section-specific responsive structures; test header, cards, forms, footer and floating actions.

Rules and invariants

No horizontal scroll for primary pages at supported widths; touch targets and type size remain usable.

Failure and recovery behaviour

Long text or missing image must not break layout; fallback stacking rules apply.

Acceptance criteria

Visual/functional tests cover phone, tablet, laptop and wide viewports across starter themes.

Dependencies and implementation notes

Component library, responsive design system.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-057 in the delivery tracker.

FR-058 - Generated-site accessibility

Priority: MUST Requirement family: Functional

Purpose

Deliver accessible navigation/content/forms and interaction.

Primary workflow

Use semantic markup, labels, focus management, contrast, alt text and meaningful link names.

Rules and invariants

Target WCAG 2.2 AA for core components; document exceptions and fix critical issues before release.

Failure and recovery behaviour

Accessibility error report identifies responsible component; release gate blocks severe violations.

Acceptance criteria

Keyboard and screen-reader acceptance path succeeds for core pages and dashboard workflows.

Dependencies and implementation notes

Design system, component tests, audit tools.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-058 in the delivery tracker.

FR-059 - Generated-site performance

Priority: MUST Requirement family: Functional

Purpose

Keep public pages fast and stable on mobile networks.

Primary workflow

Use responsive images, lazy load below-fold media, limit JS and fonts, reserve dimensions and cache immutable releases.

Rules and invariants

Five-image hero is an optional variant, not a reason to preload large full-resolution assets; avoid layout shifts.

Failure and recovery behaviour

Slow media uses placeholder/reserved dimensions and does not block main content.

Acceptance criteria

Lighthouse/lab and field metrics are collected; regression gate enforces agreed budgets for representative sites.

Dependencies and implementation notes

Renderer, Cloudinary transformations, performance CI.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-059 in the delivery tracker.

FR-060 - Custom hostname routing

Priority: MUST Requirement family: Functional

Purpose

Route verified custom hostnames to the matching published site.

Primary workflow

Normalise host, resolve against active verified mapping and render only the associated release.

Rules and invariants

Unknown host returns safe not-found; hostname mapping is unique and cannot target arbitrary tenant; trust proxy headers only from approved infrastructure.

Failure and recovery behaviour

Lookup mismatch or stale mapping disables serving rather than guessing a tenant.

Acceptance criteria

Tests confirm verified host routes correctly and unknown/removed host cannot reveal site content.

Dependencies and implementation notes

Host resolver, DomainMapping table, hosting provider.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-060 in the delivery tracker.

FR-061 - Domain ownership verification

Priority: MUST Requirement family: Functional

Purpose

Prove control of a customer domain before serving site content there.

Primary workflow

Issue supported DNS/HTTP challenge; poll provider/records; mark verified only after trusted proof.

Rules and invariants

Do not use registration payment or email alone as proof unless registrar/provider policy explicitly defines it; challenge expires.

Failure and recovery behaviour

Stale/failed challenge remains pending and gives instructions to retry.

Acceptance criteria

Unverified domain cannot activate mapping or receive a published customer site.

Dependencies and implementation notes

Registrar/host provider adapter, DNS resolver.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-061 in the delivery tracker.

FR-062 - Domain availability search

Priority: MUST Requirement family: Functional

Purpose

Search availability and quote price for supported domains.

Primary workflow

Normalise requested domain and TLD, query registrar adapter, return timestamped availability and pricing/renewal details where available.

Rules and invariants

Do not imply availability is guaranteed until registration completes; display unsupported extension as unsupported.

Failure and recovery behaviour

Provider timeout shows retryable result, not “available”; stale quotes expire.

Acceptance criteria

Known test domains return correctly mapped statuses; unsupported TLD cannot proceed to checkout.

Dependencies and implementation notes

Registrar adapter, price book, cache policy.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-062 in the delivery tracker.

FR-063 - Domain registration

Priority: MUST Requirement family: Functional

Purpose

Register a selected domain for the customer when payment is verified.

Primary workflow

Collect required registrant details, verify eligibility, create order, process payment, enqueue idempotent registrar request and persist provider IDs/dates.

Rules and invariants

Domain should be registered under appropriately disclosed customer registrant details; line item includes term and renewal price; no duplicate purchase on retries.

Failure and recovery behaviour

Uncertain registrar response triggers lookup/reconciliation before retry; charge-but-fail enters recovery/refund workflow.

Acceptance criteria

Simulated duplicate job cannot register twice; customer sees verified registrar outcome and receipt.

Dependencies and implementation notes

Payment order, registrar adapter, workflow/idempotency.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-063 in the delivery tracker.

FR-064 - DNS setup and TLS readiness

Priority: MUST Requirement family: Functional

Purpose

Configure or guide DNS and report actual connection status.

Primary workflow

Apply supported records or provide exact customer DNS instructions; poll hostname/provider state and HTTPS health.

Rules and invariants

Do not mark live until mapping, DNS/provider state and HTTPS check are successful; DNS/TLS are distinct from domain registration.

Failure and recovery behaviour

Timeout remains pending with last checked time; provider errors lead to retry/support state.

Acceptance criteria

End-to-end request to custom hostname returns correct release over HTTPS before active status.

Dependencies and implementation notes

Hosting provider API, DNS checks, TLS state machine.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-064 in the delivery tracker.

FR-065 - Domain renewal reminders

Priority: MUST Requirement family: Functional

Purpose

Track expiry/renewal state and notify owners.

Primary workflow

Persist registrar renewal date, queue reminder schedule and record delivery; auto-renew only when explicitly configured and supported.

Rules and invariants

Show renewal price/date and customer responsible payment method; failed renewals escalate before expiry.

Failure and recovery behaviour

If registrar data cannot be retrieved, flag uncertain status and alert operations, not silently assume renewed.

Acceptance criteria

Tests cover reminder timing, deduplication, changed expiry date and failed renewal escalation.

Dependencies and implementation notes

Registrar sync, scheduler/workflows, Brevo.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-065 in the delivery tracker.

FR-066 - Domain transfer and portability

Priority: MUST Requirement family: Functional

Purpose

Allow customers to understand and execute domain transfer/ownership changes.

Primary workflow

Provide documented transfer-out instructions or supported API flow; verify account/registrant and track transfer state.

Rules and invariants

Do not block export or retain a domain to force hosting retention; disclose provider limitations and required unlock/auth code path.

Failure and recovery behaviour

Provider-specific manual process gives clear status and support path.

Acceptance criteria

A customer can locate required domain details and initiate supported transfer workflow without losing unrelated site data.

Dependencies and implementation notes

Registrar adapter, account ownership policy, support UI.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-066 in the delivery tracker.

FR-067 - Checkout creation

Priority: MUST Requirement family: Functional

Purpose

Create a server-priced checkout for subscription, domain or add-on.

Primary workflow

Calculate price/line items server-side, snapshot plan and tax/term values, request provider checkout and attach stable internal order ID.

Rules and invariants

Ignore client-submitted amounts; show renewal details and included AI allowance before confirmation.

Failure and recovery behaviour

Provider failure leaves a recoverable unpaid order; repeated click reuses or replaces intent safely.

Acceptance criteria

Tampered client price cannot change final amount and checkout corresponds to displayed terms.

Dependencies and implementation notes

Plan service, price book, payment adapter.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-067 in the delivery tracker.

FR-068 - Payment webhook processing

Priority: MUST Requirement family: Functional

Purpose

Reconcile payment status using trusted provider events.

Primary workflow

Verify authenticity, store unique event ID, retrieve payment state if required, update ledger transactionally and enqueue fulfillment.

Rules and invariants

Browser redirect is not payment authority; duplicate events are idempotent; invalid signatures rejected.

Failure and recovery behaviour

Out-of-order events are reconciled against provider truth and valid state transitions.

Acceptance criteria

Replay and duplicated webhook do not duplicate subscription or domain fulfilment.

Dependencies and implementation notes

Payment provider SDK/API, database unique keys, background workflow.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-068 in the delivery tracker.

FR-069 - Plan entitlements

Priority: MUST Requirement family: Functional

Purpose

Apply plan feature and resource limits consistently.

Primary workflow

Read current entitlement snapshot for site count, page count, export, custom domains, usage and support; enforce in UI and server.

Rules and invariants

Server is authoritative; changes in plan version are versioned; downgrade effects are disclosed.

Failure and recovery behaviour

Entitlement-service outage uses safe fallback for critical reads and does not grant privileged paid features.

Acceptance criteria

API tests cannot exceed configured quotas by bypassing UI or racing concurrent requests.

Dependencies and implementation notes

Plan catalogue, subscription lifecycle, usage ledger.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-069 in the delivery tracker.

FR-070 - AI usage allowance

Priority: MUST Requirement family: Functional

Purpose

Track AI usage and expose an understandable allowance to customers.

Primary workflow

Record job type/provider usage and apply product-defined unit weight against period allowance; show remaining balance and reset date.

Rules and invariants

Platform-caused failed jobs follow a documented refund/release rule; actual provider costs are recorded separately from user-facing credits.

Failure and recovery behaviour

Duplicate callback cannot double-debit; allowance exhaustion prevents new chargeable generation while preserving read/export access as policy permits.

Acceptance criteria

Each generation has one usage ledger event; displayed balance matches ledger.

Dependencies and implementation notes

AI usage events, subscription entitlement, billing UI.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-070 in the delivery tracker.

FR-071 - Invoices and receipts

Priority: MUST Requirement family: Functional

Purpose

Make charges, line items, terms and renewal information visible.

Primary workflow

Create invoice/receipt from verified provider transaction and frozen order details; expose downloadable or viewable history.

Rules and invariants

Amounts stored as integer minor units with currency; domain and hosting items are distinct; refund/credit events are traceable.

Failure and recovery behaviour

Receipt generation failure does not change payment truth; retry is safe.

Acceptance criteria

Customer can reconcile receipt with provider payment and plan/domain term.

Dependencies and implementation notes

Order/invoice model, payment webhooks, email.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-071 in the delivery tracker.

FR-072 - Transactional email delivery

Priority: MUST Requirement family: Functional

Purpose

Send verification, reset, build, publish, payment and domain emails.

Primary workflow

Use versioned branded templates via Brevo or selected provider; capture message status/bounce metadata where available.

Rules and invariants

No secrets/credentials in messages; verified sender domain and provider API credentials server-side.

Failure and recovery behaviour

Transient provider failure retries with idempotency; bounced address is flagged without breaking in-app notification.

Acceptance criteria

Templates render in common clients and events are not duplicated after job retry.

Dependencies and implementation notes

Email provider adapter, templates, notification jobs.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-072 in the delivery tracker.

FR-073 - Newsletter opt-in

Priority: MUST Requirement family: Functional

Purpose

Allow users to subscribe to platform newsletters with recorded consent.

Primary workflow

Record opt-in source, timestamp, policy/template version and destination; use double opt-in if adopted by policy/market.

Rules and invariants

Marketing consent separate from service messages; checkbox not preselected; consent can be withdrawn.

Failure and recovery behaviour

Provider/list sync failure remains queued and is observable; UI does not falsely confirm subscription before policy requires it.

Acceptance criteria

Consent evidence is available and unsubscribed users are not added to new campaigns.

Dependencies and implementation notes

Brevo contact/list integration, consent model.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-073 in the delivery tracker.

FR-074 - Newsletter unsubscribe and suppression

Priority: MUST Requirement family: Functional

Purpose

Respect marketing unsubscribe and provider suppression.

Primary workflow

Process signed/unsubscribe link and provider webhook/list state; store suppression and prevent future marketing campaign sends.

Rules and invariants

Transactional message classification cannot be abused to bypass marketing consent; resubscription requires deliberate valid action.

Failure and recovery behaviour

Provider sync delay is handled by local suppression first and later reconciliation.

Acceptance criteria

An unsubscribed recipient receives no subsequent marketing blast in tests, including queued campaigns.

Dependencies and implementation notes

Brevo, consent/suppression database, campaign jobs.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-074 in the delivery tracker.

FR-075 - In-app notification lifecycle

Priority: MUST Requirement family: Functional

Purpose

Persist and manage notifications independently from email.

Primary workflow

Create from domain events using idempotency key; support unread/read and destination navigation.

Rules and invariants

Notification access limited to recipient; event content avoids secrets and sensitive details.

Failure and recovery behaviour

Broken target route shows safe fallback; email failure does not delete in-app item.

Acceptance criteria

Repeated event creates one notification, and read status persists across devices.

Dependencies and implementation notes

Notification service, event bus, authz.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-075 in the delivery tracker.

FR-076 - Workflow retry and resume

Priority: MUST Requirement family: Functional

Purpose

Recover from transient failures without restarting all successful stages.

Primary workflow

Persist job steps and outputs, classify failures, retry with bounded exponential backoff and jitter.

Rules and invariants

No infinite retry; permanent failures require targeted repair/manual action; retry policy varies by operation type.

Failure and recovery behaviour

Worker crash resumes after completed stage; if provider response is uncertain, reconcile before side-effect retry.

Acceptance criteria

Fault-injection tests verify no lost output or duplicate charge/register/publish side effects.

Dependencies and implementation notes

Durable workflow engine, job store, idempotency ledger.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-076 in the delivery tracker.

FR-077 - Idempotency for external actions

Priority: MUST Requirement family: Functional

Purpose

Prevent duplicated payments, registration and publishing from retries.

Primary workflow

Assign deterministic or persisted idempotency key and unique internal action row; query provider state when outcome is uncertain.

Rules and invariants

Never generate a new key merely because a request timed out if the first request may have succeeded.

Failure and recovery behaviour

Unknown outcome moves to reconciliation state; support can inspect evidence and safely resume.

Acceptance criteria

Repeated identical requests lead to one external effect and consistent internal status.

Dependencies and implementation notes

Order/service IDs, provider APIs, unique constraints.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-077 in the delivery tracker.

FR-078 - Build progress reporting

Priority: MUST Requirement family: Functional

Purpose

Show stage-by-stage status that survives refresh and device change.

Primary workflow

Expose persisted job state, progress labels, last update, warnings and retry action where permitted.

Rules and invariants

Progress percentages must not imply false precision; completed stage only shown after persisted completion.

Failure and recovery behaviour

Stalled queue or lost connection shows last-known status and refresh path.

Acceptance criteria

Refreshing during a build returns the same job and current state without duplicate generation.

Dependencies and implementation notes

Generation job store, UI polling/subscription.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-078 in the delivery tracker.

FR-079 - Concurrency and AI cost controls

Priority: MUST Requirement family: Functional

Purpose

Protect service quality and unit economics from bursts and abuse.

Primary workflow

Use provider/global/site/account concurrency caps, per-operation quotas, spend alerts and fair scheduling.

Rules and invariants

Limits and errors are user-safe; do not let one tenant starve others; emergency kill switch pauses new expensive work.

Failure and recovery behaviour

Limit reached queues or rejects with retry guidance; does not corrupt completed jobs.

Acceptance criteria

Load test confirms configured cap and cost ceiling behaviour; alert triggers on defined anomaly.

Dependencies and implementation notes

Usage ledger, job queue, plan service, monitoring.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-079 in the delivery tracker.

FR-080 - Static export generation

Priority: MUST Requirement family: Functional

Purpose

Compile canonical config into a usable static website package.

Primary workflow

Freeze input versions; generate route HTML/assets/styles/scripts; generate sitemap and README; test package with a local static server.

Rules and invariants

Only registered export-compatible components; output does not require platform runtime for declared static features.

Failure and recovery behaviour

Unsupported component fails or yields an explicit warning/degradation decision; broken package never offered as successful.

Acceptance criteria

Exported site loads independently at supported static host, with routes/media verified.

Dependencies and implementation notes

Export compiler, section manifests, asset catalogue.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-080 in the delivery tracker.

FR-081 - Export dependency report

Priority: MUST Requirement family: Functional

Purpose

Disclose backend-dependent or restricted features before export.

Primary workflow

Scan configured components/integrations for server/API dependencies and produce supported/external/unsupported list with resolution notes.

Rules and invariants

Do not claim forms, newsletter, database, auth or live features work if they need backend.

Failure and recovery behaviour

Missing dependency mapping prevents a false “fully portable” status.

Acceptance criteria

Report correctly flags each dependency in test fixtures and appears in ZIP/readme before download.

Dependencies and implementation notes

Manifest export capability, integration registry.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-081 in the delivery tracker.

FR-082 - Secure export download

Priority: MUST Requirement family: Functional

Purpose

Deliver package privately and safely.

Primary workflow

Queue export, store artifact in private object storage, issue short-lived signed URL after ownership check.

Rules and invariants

No public permanent bucket path; apply size/expiry limits and scan artifacts for secrets.

Failure and recovery behaviour

Expired link can be reissued after ownership check; failed scan blocks download.

Acceptance criteria

Another tenant cannot fetch export; expired signed URL no longer works.

Dependencies and implementation notes

Object storage, authz, export worker, secret scanner.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-082 in the delivery tracker.

FR-083 - Connected external deployment

Priority: LATER Requirement family: Functional

Purpose

Allow customers to connect supported external hosting/Git integration.

Primary workflow

Use OAuth/deployment adapter, record external project/repo mapping and push a specific release through an approved workflow.

Rules and invariants

Least-privilege scopes; never store raw tokens in user-readable records; disconnect/revoke supported.

Failure and recovery behaviour

Provider outage or rejected deploy leaves prior external release intact and shows logs/status.

Acceptance criteria

Successful deploy maps to known release; disconnect stops future deployment without deleting platform source.

Dependencies and implementation notes

External hosting providers, OAuth, release model.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-083 in the delivery tracker.

FR-084 - Customer data portability

Priority: MUST Requirement family: Functional

Purpose

Provide access to customer-owned content and key account records.

Primary workflow

Create structured export of business brief, pages, blog drafts, media references, theme/site config and invoices subject to policy.

Rules and invariants

Exclude internal prompts, platform secrets, other tenant information and nontransferable licensed assets.

Failure and recovery behaviour

Large export is asynchronous; incomplete data is clearly listed.

Acceptance criteria

Export contains own supported records, schema/version documentation and no secrets.

Dependencies and implementation notes

Data inventory, export job, privacy workflow.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-084 in the delivery tracker.

FR-085 - Backup and restore

Priority: MUST Requirement family: Functional

Purpose

Maintain recoverable backups and test restoration.

Primary workflow

Schedule database backups and appropriate media/config backups; track age/status; conduct restore drills.

Rules and invariants

Backup access restricted; backups encrypted and retention documented; restore must include app/schema compatibility.

Failure and recovery behaviour

Backup job failure alerts; recovery runbook names owner and expected steps.

Acceptance criteria

Quarterly or agreed cadence restore test meets defined RPO/RTO and records evidence.

Dependencies and implementation notes

DB provider, object storage, operations runbook.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-085 in the delivery tracker.

FR-086 - Audit logging

Priority: MUST Requirement family: Functional

Purpose

Record privileged, financial, domain and publish actions.

Primary workflow

Append actor, action, resource, timestamp, request/job ID, outcome, reason and safe before/after summary.

Rules and invariants

No secrets or unnecessary full content; standard users cannot edit/delete audit events.

Failure and recovery behaviour

Audit sink failure on critical operation follows defined safe policy and generates urgent alert.

Acceptance criteria

Publish rollback, domain change, refund, admin access and prompt rollout can be reconstructed from events.

Dependencies and implementation notes

Audit schema, identity, critical service hooks.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-086 in the delivery tracker.

FR-087 - Admin operations console

Priority: MUST Requirement family: Functional

Purpose

Give authorised staff controlled tools to support and operate the platform.

Primary workflow

View queues/errors, assets, manifests, prompt releases, plans, domain state and account/site diagnostics; permit approved actions.

Rules and invariants

RBAC, least privilege, reason capture and audit for writes or private data access; no blanket impersonation by default.

Failure and recovery behaviour

Admin permission error denies action; failed provider operation remains traceable and does not fake success.

Acceptance criteria

Role tests show curator cannot change billing, support cannot access secrets, and admin actions are audited.

Dependencies and implementation notes

RBAC, audit log, operations APIs.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-087 in the delivery tracker.

FR-088 - Prompt and model registry

Priority: MUST Requirement family: Functional

Purpose

Version prompts, schemas, model/provider configuration and controlled rollout.

Primary workflow

Register prompt ID/version, stage, schema version, provider/model eligibility, evaluation set and feature flag.

Rules and invariants

Only approved versions serve production; secret/provider configuration stored separately.

Failure and recovery behaviour

Bad prompt can be rolled back without changing site data/config; in-flight jobs pin prompt version.

Acceptance criteria

Canary benchmark compares validity, factuality, quality, cost and repair rate before promotion.

Dependencies and implementation notes

AI adapter, registry, evaluation harness.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-088 in the delivery tracker.

FR-089 - Product analytics and observability

Priority: MUST Requirement family: Functional

Purpose

Measure operational performance and product outcomes without over-collecting content.

Primary workflow

Emit structured events and metrics for activation, preview, publish, job latency/cost, domain, payment, export and errors.

Rules and invariants

Use pseudonymous IDs where practical; redact prompts/PII; control access and retention.

Failure and recovery behaviour

Telemetry outage must not block user-critical operations; missing metrics are detected.

Acceptance criteria

Dashboards can identify failing stage/provider, site-level cost trends and successful funnel events.

Dependencies and implementation notes

Logging/metrics/tracing stack, event schema.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-089 in the delivery tracker.

FR-090 - Terms acceptance and policy versioning

Priority: MUST Requirement family: Functional

Purpose

Make subscription, domain, privacy and acceptable-use terms clear and auditable before purchase or material account changes.

Primary workflow

Present relevant terms and renewal details at signup/checkout, store policy version and acceptance evidence, and surface material changes through an explicit notice/acceptance flow where required.

Rules and invariants

Do not use acceptance of marketing consent as a condition of service where not justified; show domain ownership, renewal price, cancellation, export and retention terms plainly.

Failure and recovery behaviour

If a required policy is unavailable or a checkout quote changes, stop fulfillment and request review/acceptance rather than silently proceeding.

Acceptance criteria

The system records who accepted which version and when; checkout presents the saved terms and renewal details; withdrawing marketing consent does not cancel core service.

Dependencies and implementation notes

Terms/policy content, checkout UI, consent/audit model, legal review.

Required test evidence

At minimum, include: (1) one successful path; (2) unauthorised/invalid input path; (3) retry or boundary case where applicable; (4) regression coverage for relevant tenant, responsive or billing invariants. Link automated test IDs and release evidence to FR-090 in the delivery tracker.

38. Data Dictionary and Relationship Notes

The following is the logical data model. Exact Prisma names/relations and column types should be normalised during implementation review. Store money in integer minor units with an explicit currency code. Use immutable snapshots for published versions and unique constraints for provider event identifiers.

Entity	Purpose	Key fields (illustrative)
User	Account identity and profile	id, email_normalised, display_name, phone_optional, email_verified_at, status, timezone, created_at
AuthIdentity	External/auth provider binding	id, user_id, provider, provider_subject, provider_email, created_at; unique(provider, provider_subject)
UserPreference	UI/email/locale settings	user_id, locale, timezone, notification_preferences_json, reduced_motion_pref
ConsentRecord	Consent evidence and withdrawal	id, user_id, consent_type, source, policy_version, consented_at, withdrawn_at
Site	Website tenant/root record	id, owner_user_id, name, platform_slug, lifecycle_status, current_draft_revision_id, published_release_id, created_at
SiteMembership	Future sharing/agency membership	id, site_id, user_id, role, invited_by, status; unique(site_id,user_id)
BusinessProfile	Approved business facts used in prompts	site_id, facts_json, source_map_json, last_confirmed_at, locale, market_context
SiteRevision	Immutable draft/config snapshot	id, site_id, version, config_json, created_by, reason, created_at
Page	Route and editorial state	id, site_id, slug, page_type, title, status, current_content_revision_id, canonical_path
PageContentRevision	Versioned field payload	id, page_id, revision, content_json, schema_version, source_job_id, created_at
SectionManifestRelease	Registry snapshot reference	id, section_id, semantic_version, manifest_hash, release_status, released_at
SectionInstance	Use of section in a page revision	id, page_revision_id, section_id, section_version, sort_order, content_ref, settings_json
ThemeDefinition	Global approved themes	id, version, token_schema, compatibility_metadata, status
SiteThemeRevision	Theme chosen for site	id, site_id, revision, theme_id, token_overrides_json, created_by
Asset	Curated media catalogue record	id, cloudinary_public_id, status, licence fields, tags, dimensions, alt_text, review metadata
AssetBinding	Asset assigned to site slot	id, site_id, section_instance_id, slot_id, asset_id, role, selected_at
Conversation	Chat thread for site	id, site_id, user_id, status, summary, created_at, updated_at
ConversationMessage	Message and action linkage	id, conversation_id, actor_type, text/content_json, operation_id, created_at
GenerationJob	Durable orchestration state	id, site_id, stage, input_version_ids, prompt_version, schema_version, status, attempt, output_ref, error_code
GenerationJobStep	Independently persisted workflow step	id, job_id, step_key, status, output_ref, idempotency_key, started_at, completed_at
AIUsageEvent	Provider usage ledger	id, user_id, site_id, job_id, provider, model, tokens, estimated_cost_minor, currency, allowance_units, outcome
PromptVersion	Approved prompt contract	id, template_key, version, stage, schema_version, model_allowlist, content_hash, rollout_status
PreviewBuild	Isolated preview artifact	id, site_id, revision_id, url_token_hash, checks_json, status, expires_at

Entity	Purpose	Key fields (illustrative)
PublishRelease	Immutable public release	id, site_id, revision_id, build_id, component_registry_hash, status, created_at, published_at, published_by
DomainMapping	Custom/platform hostname association	id, site_id, hostname_normalised, type, verified_at, tls_status, mapping_status, provider_ref
RegisteredDomain	Registrar-side domain record	id, user_id, site_id, domain, registrar, registrar_ref, term_start, expires_at, renewal_status, transfer_status
DomainOperation	Idempotent domain action	id, registered_domain_id, operation, idempotency_key, state, provider_response_ref, last_error
PlanDefinition	Versioned plan/entitlements	id, version, interval, price_minor, currency, entitlements_json, published_at
Subscription	Customer plan lifecycle	id, user_id, plan_id, provider, provider_subscription_ref, status, period_start, period_end, cancel_at_period_end
Order	Frozen checkout intent	id, user_id, status, idempotency_key, line_items_json, subtotal_minor, tax_minor, total_minor, currency
Invoice	Payment/accounting record	id, order_id, user_id, provider_invoice_ref, amount_minor, currency, status, issued_at, paid_at
PaymentEvent	Verified provider payment/webhook record	id, provider, provider_event_id, event_type, payload_hash, payment_ref, verified_at, processed_at
UsageLedgerEntry	Customer allowance debit/credit	id, user_id, subscription_id, operation_ref, units_delta, reason, idempotency_key, created_at
EmailJob	Transactional/marketing email job	id, user_id, category, template_version, provider_ref, recipient_hash, status, scheduled_at
Notification	In-app notification	id, user_id, site_id_optional, type, title, body, target_route, read_at, idempotency_key
AuditEvent	Append-only privileged/action record	id, actor_type, actor_id, action, resource_type, resource_id, request_id, reason, safe_diff_json, created_at
ExportJob	Export request and artifact	id, site_id, revision_id, export_type, status, dependency_report_json, artifact_ref, expires_at
ExternalDeployment	Future connected hosting integration	id, site_id, provider, external_project_ref, token_secret_ref, connected_by, status, last_deploy_release_id
SupportCase	Customer support issue	id, user_id, site_id_optional, category, status, priority, summary, assigned_to, created_at

38.1 High-value constraints and indexes

- Unique normalised email where the auth model requires it; unique `(provider, provider_subject)` for identities.
- Unique normalised hostname across active mappings; indexed hostname lookup and `status`.
- Unique `(site_id, slug)` for current page routes; explicit redirect table for retired slugs.
- Unique provider event ID and idempotency keys within their defined scope.
- Indexes on `site_id`, ownership/member relationship, `created_at`, status, due timestamps and foreign keys used in job and dashboard lists.
- Restrict update/delete on published release and audit event tables; use application-level append-only patterns and DB privileges.
- Use JSONB for flexible configuration payloads only when schema versioned and validated; keep query-critical/unique fields in normal columns.

39. API and Event Catalogues

39.1 Representative endpoint catalogue

Method	Path	Purpose	Gate / notes
POST	/api/auth/register	Create pending account	Public, rate limited
POST	/api/auth/verify-email	Redeem verification token	Token validation
POST	/api/auth/sign-in	Start session	Auth provider
POST	/api/sites	Create site	Signed-in, quota check
GET	/api/sites	List own sites	Owner/membership query
PATCH	/api/sites/{siteId}/brief	Update business facts	Site owner; schema validation
POST	/api/sites/{siteId}/conversations	Create/continue chat	Site owner
POST	/api/sites/{siteId}/messages	Submit instruction	Site owner; rate limit and intent routing
POST	/api/sites/{siteId}/generation-jobs	Start/restart generation stage	Site owner; allowance and idempotency
GET	/api/generation-jobs/{jobId}	Get persisted job status	Owner of associated site
GET	/api/sites/{siteId}/revisions	List revisions	Site owner
POST	/api/sites/{siteId}/preview-builds	Build a preview	Owner; build quota
POST	/api/sites/{siteId}/publish	Publish an approved revision	Owner; preflight passed
POST	/api/sites/{siteId}/rollback	Restore a prior release	Owner/admin; confirm action
GET	/api/assets/search	Search approved images	Public metadata subset or signed-in; no private licence notes exposed
POST	/api/sites/{siteId}/domains	Attach a domain	Owner; verification workflow
POST	/api/domains/{domainId}/verify	Check ownership/TLS state	Owner; throttled
POST	/api/billing/checkout	Create provider checkout	Owner; server-priced
GET	/api/billing/usage	Return allowance and metered summary	Authenticated owner
POST	/api/sites/{siteId}/exports	Start export job	Owner; plan and quota
GET	/api/exports/{exportId}	Get export status/report	Owner
POST	/api/exports/{exportId}/download-url	Issue short-lived URL	Owner; ready artifact only
GET	/api/notifications	List notifications	Current user
POST	/api/notifications/{id}/read	Mark read	Recipient only
POST	/api/webhooks/payment-provider	Process payment event	Signature/authenticity required
POST	/api/webhooks/email-provider	Process bounce/unsubscribe events	Provider authenticity required

39.2 Standard error envelope

```
{
  "error": {
    "code": "SITE_REVISION_CONFLICT",
    "message": "This website changed in another session. Refresh and try again.",
    "retryable": false,
    "requestId": "req_...",
    "fieldErrors": []
  }
}
```

The exact JSON format can be refined, but all APIs should use consistent status codes and safe messages. Internal stack traces and provider credentials never reach the browser.

39.3 Domain events

Event key	Producer	Meaning	Minimum payload
user.email_verified	Auth service	Account verified	Account ID and timestamp only
site.created	Site service	Site created	Site ID, owner ID, request ID
generation.job_started	Orchestrator	Stage queued	Job ID, stage, site ID
generation.stage_succeeded	Orchestrator	Validated stage saved	Job ID, stage, schema/prompt version
generation.job_failed	Orchestrator	Retry policy exhausted	Job ID, error class, retryability
preview.ready	Build service	Preview checks complete	Build ID, revision, check summary
publish.activated	Publishing	Release pointer changed	Release ID, site ID, actor
domain.registration_confirmed	Registrar adapter	Registrar confirmed registration	Domain ID, provider ref, renewal date
domain.hostname_active	Domain service	DNS/hostname/TLS validated	Mapping ID, status snapshot
domain.renewal_due	Scheduled workflow	Renewal threshold reached	Domain ID, dates and price snapshot
payment.verified	Payment adapter	Verified successful payment	Order/payment ref, amount, currency
subscription.changed	Billing service	Entitlement state changed	Subscription ID, plan/version/state
usage.ledger_posted	Usage service	Allowance ledger changed	User/site ID, operation ID, units delta
export.ready	Export worker	Artifact validated	Export ID, checksum, dependency report ID
email.bounced	Email adapter	Delivery failed	Message ID/category; minimum recipient metadata
newsletter.unsubscribed	Consent service	Marketing suppression recorded	Contact/user reference, timestamp and source
asset.licence_expiring	Asset scheduler	Review date approaching	Asset ID, deadline, permitted actions
security.tenant_access_denied	Security telemetry	Access check failed	Actor ID, resource type/id, request ID

39.4 Event handling rules

Events are versioned, persisted when delivery reliability matters, and idempotently consumed. Event consumers must tolerate duplicates and reordered delivery. Never place payment credentials, auth tokens, full prompt transcripts or unnecessary personal information in event payloads. A durable event record should contain enough to retry work without reconstructing state from a browser session.

40. Reference Configurations and Prompt Examples

40.1 Example section manifest

```
{
  "id": "services.card-grid",
  "version": "1.2.0",
  "label": "Services card grid",
  "pageTypes": ["home", "services"],
  "industries": ["general", "professional-services", "tourism"],
  "layoutType": "responsive-card-grid",
  "contentSchemaVersion": "services-card-grid.v2",
  "contentSchema": {
    "heading": {"type": "string", "minWords": 3, "maxWords": 9},
    "intro": {"type": "string", "minWords": 20, "maxWords": 45},
    "items": {
      "type": "array", "minItems": 3, "maxItems": 6,
      "item": {
        "title": {"type": "string", "maxWords": 6},
        "description": {"type": "string", "minWords": 18, "maxWords": 36},
        "ctaLabel": {"type": "string", "maxChars": 26},
        "targetRoute": {"type": "registered-internal-route"}
      }
    }
  },
  "mediaSlots": [],
  "animationPresets": ["fade-up", "stagger-cards"],
  "accessibility": {
    "semanticList": true,
    "focusVisible": true,
    "reducedMotionFallback": true
  },
  "export": {"staticSafe": true, "dependencies": []},
  "status": "released"
}
```

Illustrative only. Production code should use actual schemas and type definitions. Avoid a manifest format that is more expressive than your runtime validator can safely support.

40.2 Example theme tokens

```
{
  "id": "warm-editorial",
  "version": "1.0.0",
  "tokens": {
    "color.primary": "#174A50",
    "color.accent": "#D6A24A",
    "color.background": "#FAF8F3",
    "color.surface": "#FFFFFF",
    "color.text": "#1D2A2A",
    "color.muted": "#586766",
    "typography.display": "ApprovedDisplayFont",
    "typography.body": "ApprovedBodyFont",
    "radius.card": "1.0rem",
    "spacing.section.desktop": "5rem",
    "motion.preset": "subtle"
  },
  "contrastChecks": "required",
  "fontAssets": "self-hosted-or-approved",
  "reducedMotion": "static"
}
```

40.3 Example site-plan prompt contract

Role: propose a site plan using only supplied data and registered IDs. Inputs: BusinessBrief, supported route types, allowed section manifest summaries, plan entitlements and composition constraints. Output: SitePlan JSON with theme ID, route objects, ordered section references and per-page generation tasks. Prohibitions: no HTML/CSS/JS output, no new component IDs, no made-up claims, no domains/payment actions, no secret disclosure. Success: schema-valid plan whose routes and section IDs pass backend validation.

40.4 Example page copy prompt contract

Role: draft fields for one page or one section. Inputs: selected section manifest, business facts with provenance, intended audience, page goal, tone, length limits, internal-route allowlist and existing approved site facts. Output: exactly the fields declared in the schema. Do not

fill missing factual values with assumptions. For missing proof points, return an explicit needs-input marker only if the schema permits it; otherwise omit the optional field. Do not generate markup or external asset URLs.

40.5 Example targeted edit contract

Input: current revision ID, target page/section IDs, requested change, schema and permitted operation set. Output: proposed change-set listing only fields/operations changed, rationale and validation warnings. The backend checks current revision matches expected revision and the user owns the site. Apply only after schema validation; show a preview for material changes. The model never receives payment credentials or privileged provider tokens.

40.6 Example export manifest

```
{
  "siteId": "site_...",
  "sourceRevision": "rev_...",
  "exportCompilerVersion": "1.0.0",
  "generatedAt": "2026-10-14T00:00:00Z",
  "routes": ["/", "/about", "/services", "/blog", "/contact"],
  "assets": [
    {"assetId": "asset_...", "mode": "local-copy", "licenceValidated": true}
  ],
  "dependencies": [
    {"feature": "contact-form", "status": "external-required", "action": "Configure an approved form endpoint"},
    {"feature": "platform-analytics", "status": "not-included", "action": "Connect an external analytics service if
required"}
  ],
  "checks": {"routes": "passed", "links": "passed", "assets": "passed", "secretScan": "passed"}
}
```

41. Risk Register and Open Decisions

ID	Risk	Likelihood	Impact	Mitigation	Owner
R-01	Brand collision / name not ownable	Medium	High	Treat Fortress/Fortress AI as provisional; domain, software and trademark checks before identity investment.	Product owner
R-02	Hosting platform pricing changes or does not fit multi-tenant economics	Medium	High	Proof of concept with pricing scenarios; provider adapter and alternative deployment plan.	Engineering / finance
R-03	`.co.ke` registrar API coverage unavailable or restricted	Medium	High	Validate supported TLD and contract before advertising auto-registration; provide manual assisted path if needed.	Product / domains
R-04	Payment method recurring support misunderstood	Medium	High	Confirm per-method subscription support in writing; model M-Pesa and cards correctly.	Finance / payments
R-05	AI invents testimonials, prices or credentials	Medium	High	Fact ledger, field-level constraints, adversarial evals, owner review and alternative sections.	AI/content lead
R-06	AI and hosting costs exceed subscription revenue	Medium	High	Measure job costs, define allowances, fair-use controls, alerts and plan contribution analysis.	Product / finance
R-07	Cross-tenant data leakage	Low/Medium	Critical	RLS/application policy tests, service scopes, threat modelling and release gate.	Engineering/security
R-08	Domain takeover or wrong tenant routing	Low/Medium	Critical	Ownership verification, mapping checks, cache purge and immediate detach runbook.	Infrastructure/security
R-09	Static export silently breaks forms/search/analytics	High	High	Explicit dependency contract, export report and independent-host smoke tests.	Export engineer
R-10	Image licence does not permit local export	Medium	High	Track export rights separately; provide remote or approved substitute; no automatic redistribution.	Asset curator/legal
R-11	Design variants look repetitive or incoherent	Medium	Medium/High	Component rubric, seeded composition and within-site constraints; sample human design review.	Design lead
R-12	Mobile performance suffers from hero carousels/animation	Medium	High	Single-image fallback, media budgets, reduced motion, lazy loading and field monitoring.	Frontend lead
R-13	Third-party AI/email/payment outage blocks onboarding	Medium	High	Durable jobs, retry states, fallback provider only if tested, operational messaging.	Infrastructure
R-14	User misunderstands ownership/export limitations	Medium	High	Plain-language terms, domain registration details, export dependency report and clear cancellation flow.	Product/legal

ID	Risk	Likelihood	Impact	Mitigation	Owner
R-15	Low-quality SEO content creates reputational harm	Medium	High	Draft articles by default, editorial quality rubric, no ranking promises and targeted human review.	Content/SEO lead
R-16	Regulatory/data transfer obligations inadequately addressed	Medium	High	Legal/privacy review, data map, processor contracts and retention/deletion workflow.	Business owner/legal
R-17	Prompt/manifest rollout breaks existing sites	Medium	High	Version pinning, compatibility test, canary, migration plan and rollback.	Engineering/design system
R-18	Webhook retries duplicate charges or domain registrations	Low/Medium	Critical	Unique event IDs, idempotency keys, provider reconciliation and fault injection.	Payments/domain lead
R-19	Cloudinary account/asset loss or licence expiry	Low/Medium	High	Asset backup plan, expired-licence monitoring, provenance metadata and restoration drills.	Asset operations
R-20	Single developer becomes operational bottleneck	High	Medium/High	Documentation, automated deployment, runbooks, staged roadmap and clear module ownership.	Founder/product owner

41.1 Decisions to close before production

Topic	Decision required	Target milestone
Brand	Confirm final name, domain strategy, trademark search and social availability.	Before public branding.
Hosting	Choose provider after per-site traffic, custom hostname cost, caching and export assessment.	Before production domain commitments.
Registrar	Select registrar and verify `.co.ke` plus intended extensions, eligibility, fees, registration/renewal API.	Before domain automation release.
Payments	Confirm provider commercial account, fees, card/M-Pesa flows, webhook support, refunds and recurrence rules.	Before billing launch.
AI provider	Benchmark candidate models on common briefs and adversarial factuality tests; choose first provider.	Before prompt rollout.
Plan pricing	Calculate unit costs and define site/page/traffic/AI allowances and domain markup/renewal disclosures.	Before public checkout.
Email domain	Configure sender subdomain, SPF/DKIM/DMARC and transactional/marketing separation.	Before signup/email campaigns.
Phone verification	Decide whether optional SMS verification is in MVP after cost and risk review.	Before auth implementation freeze.
Exports	Define exactly which sections/features are static-safe and if self-contained image export is permitted.	Before export beta.
Data/legal	Review privacy notice, customer terms, domain ownership, subscription cancellation, refund and retention policy.	Before public launch.

Topic	Decision required	Target milestone
Operations	Set numeric SLO, on-call owner, response channel, data retention, backup schedule and recovery tests.	Before production support.
MVP industry focus	Choose the first three supported industries and build evaluation briefs for them.	Before section content library is final.

42. Runbooks, Checklists and Glossary

42.1 Release checklist

- No unresolved critical security or data isolation failures.
- Prompt/manifest schemas validate and required component versions are present.
- All required public routes pass link, metadata, mobile and accessibility checks.
- Hosted publish and rollback are tested against a nonproduction site.
- Static export passes external-host smoke test and includes dependency report.
- Payment webhook replay and duplicate registrar response tests pass.
- Cloudinary production assets have licence/source metadata and expiry review where needed.
- Brevo sender authentication, transactional templates and unsubscribe flow are tested.
- Subscription plan limits, usage unit wording and renewal disclosures are final and reflected in server enforcement.
- Backup restore and incident runbooks have named owners.
- Monitoring/alerts cover AI spend, job lag, domain/TLS errors, payment webhook lag and published-site health.
- Customer-facing privacy, terms, support and data portability paths are accessible.

42.2 Manual launch smoke test

62. Create test account, verify email, sign out/in and reset password.
63. On mobile, build a sample service-business site from short intake and confirm only essential follow-ups appear.
64. Inspect homepage, About, Services, Blog index, three draft articles and Contact at small phone/laptop/wide desktop sizes.
65. Change a homepage heading and one image only; confirm unrelated content remains unchanged.
66. Verify no invented testimonials/prices/awards; inspect alt text and contact details.
67. Publish to platform subdomain; check status, HTTPS, sitemap, robots, canonicals and indexability.
68. Attach controlled test domain and ensure pending DNS/TLS is not represented as active prematurely.
69. Run sandbox checkout and replay webhook; confirm one fulfillment and one invoice/usage update.
70. Export a static bundle, unpack it on a separate static host, click every core route and inspect dependency report.
71. Trigger a failed-generation retry and a publish rollback; ensure progress persists and prior live version remains available.

42.3 Operational dashboard minimum

Panel	Minimum measures
Generation health	Queue age, active jobs, success/repair/failure rates, stage p50/p95, cost by provider/stage, retries.
Publishing health	Publish success/failure, releases per hour, rollback count, public HTTP status and cache purge errors.
Domain operations	Registrations pending/failed, DNS/TLS duration, renewal dates, registrar API errors, uncertain outcomes.
Billing integrity	Webhook lag, mismatched payments, duplicate event count, unpaid fulfillment attempts, refunds/chargebacks.
Assets/export	Unavailable assets, licence expiry, export failure/size/time, dependency report failure, secret-scan findings.
Email	Transactional success/bounce, verification resend abuse, campaign unsubscribe/complaint and suppressed contacts.
Security	Cross-tenant denials, login/OTP spikes, unusual admin actions, rate-limit hits and secret-scanner alerts.

42.4 Glossary

Term	Definition
AI allowance	Customer-facing unit budget for generation/editing; distinct from raw provider tokens and cost.
Asset binding	Record connecting a registered media asset to a particular section media slot.

Term	Definition
Canonical configuration	Structured, versioned representation of a site used by both hosted rendering and export.
Composition engine	Rules-based service selecting compatible sections, theme and order.
Custom hostname/domain mapping	Association between a verified hostname and a platform site.
Durable workflow	A long-running job whose steps and results survive worker restart and safely retry.
Idempotency	Guarantee that repeating the same logical operation does not cause duplicate side effects.
Manifest	Versioned metadata contract describing a section's schema, compatibility, media, motion, accessibility and export rules.
Multi-tenancy	Multiple logically isolated customer sites/users using shared application infrastructure.
Published release	Immutable approved website version served publicly.
RLS	PostgreSQL Row Level Security; database policies controlling which rows can be read/written under a DB identity.
Static export	Portable HTML/CSS/JS/asset package without a required platform runtime for its declared supported features.
Tenant isolation	Controls preventing one account from reading or modifying another account's resources.
TLS/HTTPS	Transport encryption and server identity for secure website delivery.
Prompt version	Immutable prompt contract version pinned to an AI job for reproducibility.
Provider adapter	Internal interface isolating application code from a particular AI, registrar, email or payment vendor.

43. Source References and Validation Notes

References below are official product or government documentation used to shape the architecture. Vendor documentation and commercial terms change; re-check the exact feature, plan limits, API coverage and contract before implementation or launch. These sources support technical design choices but do not replace a proof of concept or legal/commercial review.

Source	URL	Use in TDD
Next.js Static Exports	https://nextjs.org/docs/app/guides/static-exports	Static HTML export model and constraints for features requiring a server.
Supabase Row Level Security	https://supabase.com/docs/guides/database/postgres/row-level-security	RLS concepts and the need to define policies and privileges carefully.
Prisma Next.js Guide	https://www.prisma.sh/docs/guides/frameworks/nextjs	Using Prisma in a Next.js application.
Cloudinary DAM metadata	https://cloudinary.com/documentation/dam_manage_metadata	Asset metadata and management concepts.
Cloudinary Upload API	https://cloudinary.com/documentation/image_upload_api_reference	Asset upload and provider API reference.
Cloudinary transformations	https://cloudinary.com/documentation/image_transformations	Image transformations and responsive delivery.
Cloudinary image optimisation	https://cloudinary.com/documentation/image_optimization	Optimisation guidance for delivered media.
OpenAI Structured Outputs	https://developers.openai.com/api/docs/guides/structured-outputs	Example of schema-constrained model responses; provider support differs.
Inngest retries and error handling	https://www.inngest.com/docs/durable-execution/guides-and-advanced/error-handling/retries	Example of durable retries and execution behaviour.
Vercel for Platforms	https://vercel.com/platforms	Multi-tenant platform hosting option; confirm commercial terms and limits.
Cloudflare for SaaS custom hostnames	https://developers.cloudflare.com/cloudflare-for-platforms/cloudflare-for-saas/domain-support/create-custom-hostnames/	Custom hostname onboarding concepts and lifecycle.
ResellerClub Developer API	https://developer.resellerclub.com/	Candidate domain reseller API; verify desired TLD coverage.
Better Auth Email/Password	https://better-auth.com/docs/authentication/email-password	Candidate authentication feature documentation.
OWASP Authentication Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html	Authentication design practices and risk controls.
Brevo transactional email	https://developers.brevo.com/docs/send-a-transactional-email	Transactional email API candidate.
IntaSend Developer Documentation	https://developers.intasend.com/	Payment collection, M-Pesa/card options and provider integration reference.
IntaSend Subscriptions	https://developers.intasend.com/guides/subscriptions/	Subscription/recurrence functionality; confirm support by payment method.
Kenya ODPC - Data subject rights	https://www.odpc.go.ke/rights-of-a-data-subject/	Local privacy compliance reference; obtain current legal review.
Kenya Industrial Property Institute	https://www.kipi.go.ke/	Kenyan trademark/intellectual property registration information.
GSAP documentation	https://gsap.com/docs/v3/	Animation API and integration reference.

Final Engineering Note

This document defines a technical baseline, not a claim that all third-party vendor features, quotas, prices, `.co.ke` registrar workflows or recurring payment methods are already confirmed. Before production, close the open decisions, run focused provider proofs of concept, review legal terms, and translate FR IDs into implementation tickets and automated tests. Preserve the core invariant: approved structured site configuration is the source of truth; hosted rendering and export are two controlled publication paths from that source.

END OF DOCUMENT Technical Design Document - MegaScript Digital AI Website Platform. Working product name remains provisional.